

Laurea Magistrale in Ingegneria Elettronica

Relazione Progettazione di Sistemi Digitali

Convolutore Isotropico

Studente

Tarantino Danilo Antonio

Matricola 247705

Professori

Perri Stefania

Spagnolo Fanny

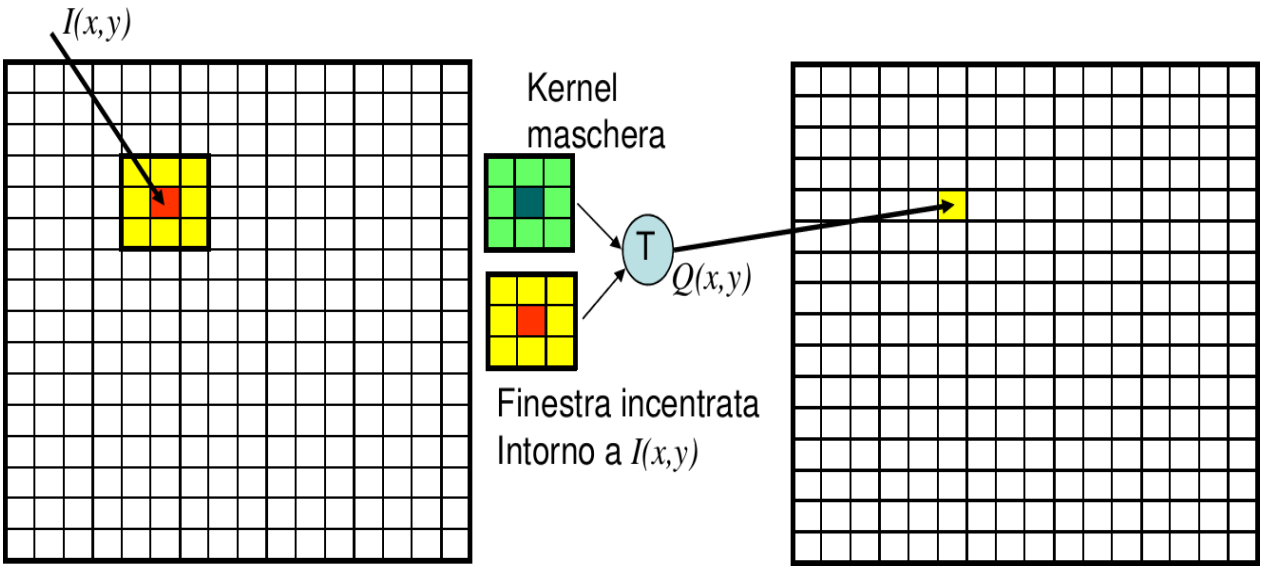
A.A. 2024/2025

Indice

1 Introduzione 3
2 Descrizione progetto 5
2.1 Buffer Line 6
2.2 Convolutore 12
2.2.1 Adder Tree 14
2.2.2 Moltiplicatore 17
2.2.3 CSA 21
2.3 Finite State Machine 23
2.4 Top Design 26
3 Simulazione 29
4 Post-Processing 39
5 Implementazione 42
6 Report e conclusioni 45

1 Introduzione

L'obiettivo di questo elaborato è di presentare la progettazione di un convolutore isotropico per il filtraggio di immagini in scala di grigi dotate di pixel a 8-bit unsigned, mediante un kernel 3x3 con coefficienti a 8-bit espressi in complemento a due. Il circuito comprende la parte di calcolo, il buffer per la formazione della finestra di pixel e una parte realtiva al controllo. Il circuito prevede l'elaborazione di immagini digitali tramite un filtraggio bidimensionale.



Per processare ogni pixel dell'immagine, si deve costruire una finestra attorno ad esso, avente le stesse dimensioni del kernel. Successivamente si eseguono i prodotti tra i pixel e i coefficienti del kernel in posizione omologa. Infine, dopo aver svolto il calcolo dei prodotti, i risultati vengono sommati per ottenere il valore del pixel nell'immagine filtrata:

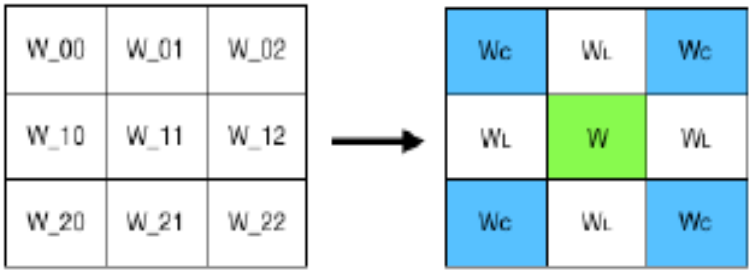
$$Q_{(i,j)} = \sum_{\substack{k=0,1,2 \\ h=0,1,2}} P_{(k+i,h+j)} \times W_{(k,h)}$$

Nel convolutore isotropico, tutti i coefficienti del filtro che hanno la stessa distanza euclidea dal coefficiente centrale hanno lo stesso valore. Quindi per un kernel 3x3 scrivo:

W per il centro;

WC per i corner;

WL per i laterali.



$$Q_{(i,j)} = P_{(i,j)} \times W + (P_{(i-1,j-1)} + P_{(i-1,j+1)} + P_{(i+1,j-1)} + P_{(i+1,j+1)}) \times W_C + (P_{(i-1,j)} + P_{(i,j+1)} + P_{(i+1,j)} + P_{(i,j+1)}) \times W_L$$

Le operazioni da eseguire consistono inizialmente nel sommare i pixel associati allo stesso coefficiente del filtro. Successivamente, i valori ottenuti vengono moltiplicati per i rispettivi coefficienti, e infine i risultati delle tre moltiplicazioni vengono sommati. Questo approccio riduce il numero totale di moltiplicazioni da effettuare.

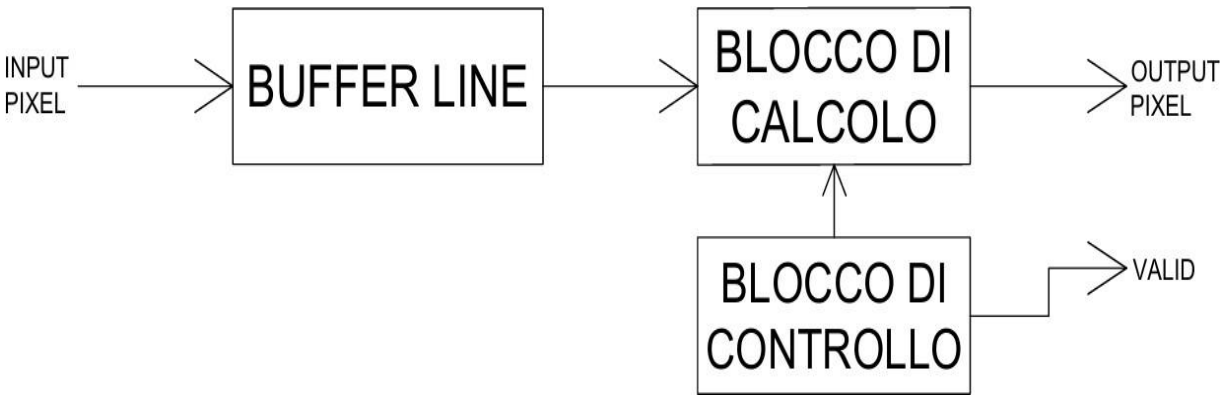
2 Descrizione progetto

Il circuito è suddiviso strutturalmente in tre macro-blocchi:

1) BUFFER LINE : consente di ottenere una finestra 3x3 costruita intorno al pixel centrale (anchor point);

2) BLOCCO DI CALCOLO : dedicato all'elaborazione dei pixel da filtrare, comprende moltiplicatori e sommatori;

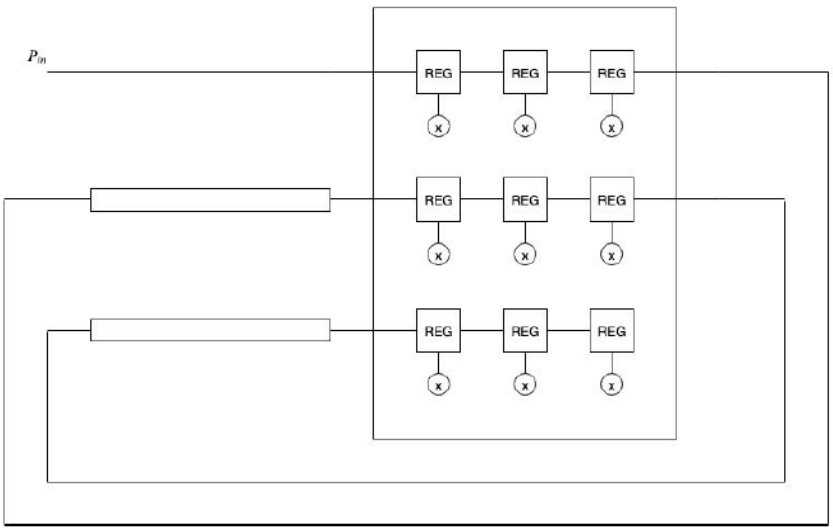
3) BLOCCO DI CONTROLLO : include la FSM per la memorizzazione dei coefficienti del kernel e la gestione del segnale Valid.



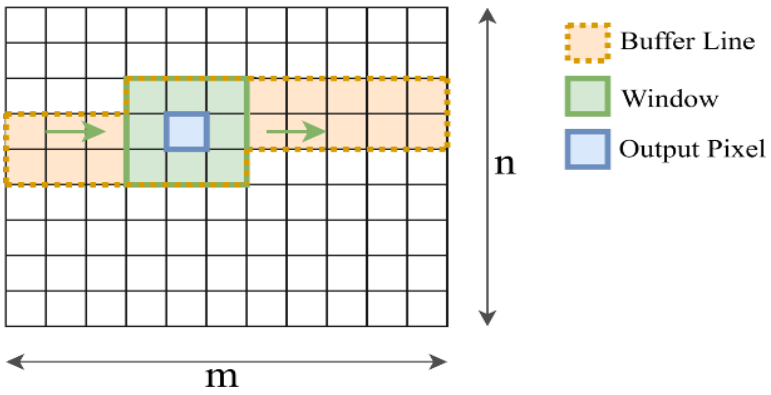
I circuiti sono progettati per elaborare un'immagine di dimensione 64x64 pixel. I bordi dell'immagine verranno esclusi, senza ricorrere a tecniche di padding. Pertanto, l'immagine prodotta in uscita avrà una dimensione di 62x62 pixel.

Partendo da un bus di ingresso di pixel a 8 bit, si costruisce la finestra 3x3, realizzata tramite una buffer line, la quale, ad ogni ciclo di clock fornisce 9 pixel a 8 bit unsigned (SIPO). Successivamente è necessario sommare i 4 pixel laterali e i 4 pixel degli angoli utilizzando due Adder Tree, dove il risultato in uscita viene rappresentato con 10 bit. Successivamente, sono state inserite 3 istanze di un moltiplicatore 10x8 bit progettato con logica "carta e penna", che realizza l'operazione tra un operando in rappresentazione binaria unsigned di 10 bit per la somma dei pixel laterali e angolari (il pixel centrale da 8 bit per poter essere moltiplicato necessita di essere esteso a 10 bit con padding di zeri a sinistra) e un operando in rappresentazione binaria signed di 8 bit per i coefficienti del filtro (che possono essere anche negativi). Il moltiplicatore producono risultati di 18 bit per il prodotto 10X8. I tre prodotti risultanti vengono poi sommati tramite un carry save a 3 ingressi, e il risultato finale è espresso su 20 bit signed. Il segnale di Valid in uscita indica i risultati validi, ed è generato da una FSM che tiene conto della latenza iniziale necessaria per riempire la prima finestra di pixel escludendo i valori ai bordi.

2.1 Buffer Line



La struttura di bufferizzazione è di tipo SIPO in quanto prende in ingresso 1 pixel per volta e restituisce parallelamente in output una finestra di 9 pixel. Essa è composta da 9 istanze del componente REG_8bit per memorizzare i pixel e due strutture FIFO realizzate con registri a scorrimento a 8 bit di profondità m-k (61) dove m è la lunghezza della riga dell'immagine (64) e k la dimensione del kernel (3). Questi SHIFT_REG mi permettono di accedere alle righe precedenti passando dal primo pixel di una riga al terzo pixel della riga precedente. Questa architettura, attraverso la concatenazione e il coordinamento di registri singoli e registri a scorrimento, consente alla finestra 3x3 di "scorrere" sull'immagine: a ogni ciclo di clock valido, un nuovo pixel entra nel sistema, e tutti i dati all'interno della finestra si spostano di una posizione, rendendo disponibile una nuova finestra 3x3 per l'elaborazione. Questo approccio basato sulla pipeline consente al sistema di eseguire operazioni di filtro in modo efficiente e parallelo. La prima finestra 3x3 di pixel si sarà formata e sarà disponibile all'uscita della BL 131 cicli di clock dopo che il primo pixel dell'immagine è stato presentato all'ingresso.



```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BL is
    Port(
        Pixel_in : in std_logic_vector(7 downto 0);
        clk, rst : in std_logic;
        P00, P01, P02, P10, P11, P12, P20, P21, P22 : out std_logic_vector(7 downto 0)
        );
end BL;

architecture Behavioral of BL is

    component REG_8BIT is
        Port(
            clk, rst : in std_logic;
            D : in std_logic_vector(7 downto 0);
            Q : out std_logic_vector(7 downto 0)
            );
    end component;

    component SHIFT_REG is
        Port(
            clk, rst : in std_logic;
            pixel_in : in std_logic_vector(7 downto 0);
            pixel_out : out std_logic_vector(7 downto 0)
            );
    end component;

    signal P00_int, P01_int, P02_int,P10_int, P11_int, P12_int,P20_int, P21_int, P22_int : std_logic_vector(7 downto 0);
    signal SR1_out, SR2_out : std_logic_vector(7 downto 0);

 begin

     REG00: REG_8BIT port map(clk => clk, rst => rst, D => Pixel_in, Q => P00_int);
     REG01: REG_8BIT port map(clk => clk, rst => rst, D => P00_int, Q => P01_int);
     REG02: REG_8BIT port map(clk => clk, rst => rst, D => P01_int, Q => P02_int);

     SHIFT1: SHIFT_REG port map(clk => clk, rst => rst, pixel_in => P02_int, pixel_out => SR1_out);

     REG10: REG_8BIT port map(clk => clk, rst => rst, D => SR1_out, Q => P10_int);
     REG11: REG_8BIT port map(clk => clk, rst => rst,  D => P10_int, Q => P11_int);
     REG12: REG_8BIT port map(clk => clk, rst => rst, D => P11_int, Q => P12_int);

     SHIFT2: SHIFT_REG port map(clk => clk, rst => rst,  pixel_in => P12_int, pixel_out => SR2_out);

     REG20: REG_8BIT port map(clk => clk, rst => rst,  D => SR2_out, Q => P20_int);
     REG21: REG_8BIT port map(clk => clk, rst => rst, D => P20_int, Q => P21_int);
     REG22: REG_8BIT port map(clk => clk, rst => rst, D => P21_int, Q => P22_int);

     P00 <= P00_int;  P01 <= P01_int;  P02 <= P02_int;
     P10 <= P10_int;  P11 <= P11_int;  P12 <= P12_int;
     P20 <= P20_int;  P21 <= P21_int;  P22 <= P22_int;

end Behavioral;
```

Codice VHDL Buffer Line

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity SHIFT_REG is
    port (
        clk, rst : in  std_logic;
        pixel_in : in  std_logic_vector(7 downto 0);
        pixel_out : out std_logic_vector(7 downto 0)
    );
end entity SHIFT_REG;

architecture Behavioral of SHIFT_REG is
    type pixel_line is array (0 to 60) of std_logic_vector(7 downto 0);
    signal pixel_buffer : pixel_line := (others => (others => '0'));
begin

    process(clk, rst)

    begin
        if rst = '1' then
            pixel_buffer <= (others => (others => '0'));

        elsif rising_edge(clk) then


                pixel_buffer(0) <= pixel_in;
                for i in 1 to 60 loop
                    pixel_buffer(i) <= pixel_buffer(i - 1);
                end loop;



        end if;

    end process;

    pixel_out <= pixel_buffer(60);

end Behavioral;
```

Codice VHDL Shift Register 61 posizioni

```vhdl
library IEEE;
use iEEE.STD_LOGIC_1164.ALL;

entity REG_8bit is
    port (
        clk,rst: in  std_logic;
        D : in  std_logic_vector(7 downto 0);
        Q : out std_logic_vector(7 downto 0)
    );
end REG_8bit;

architecture Behavioral of REG_8bit is

begin

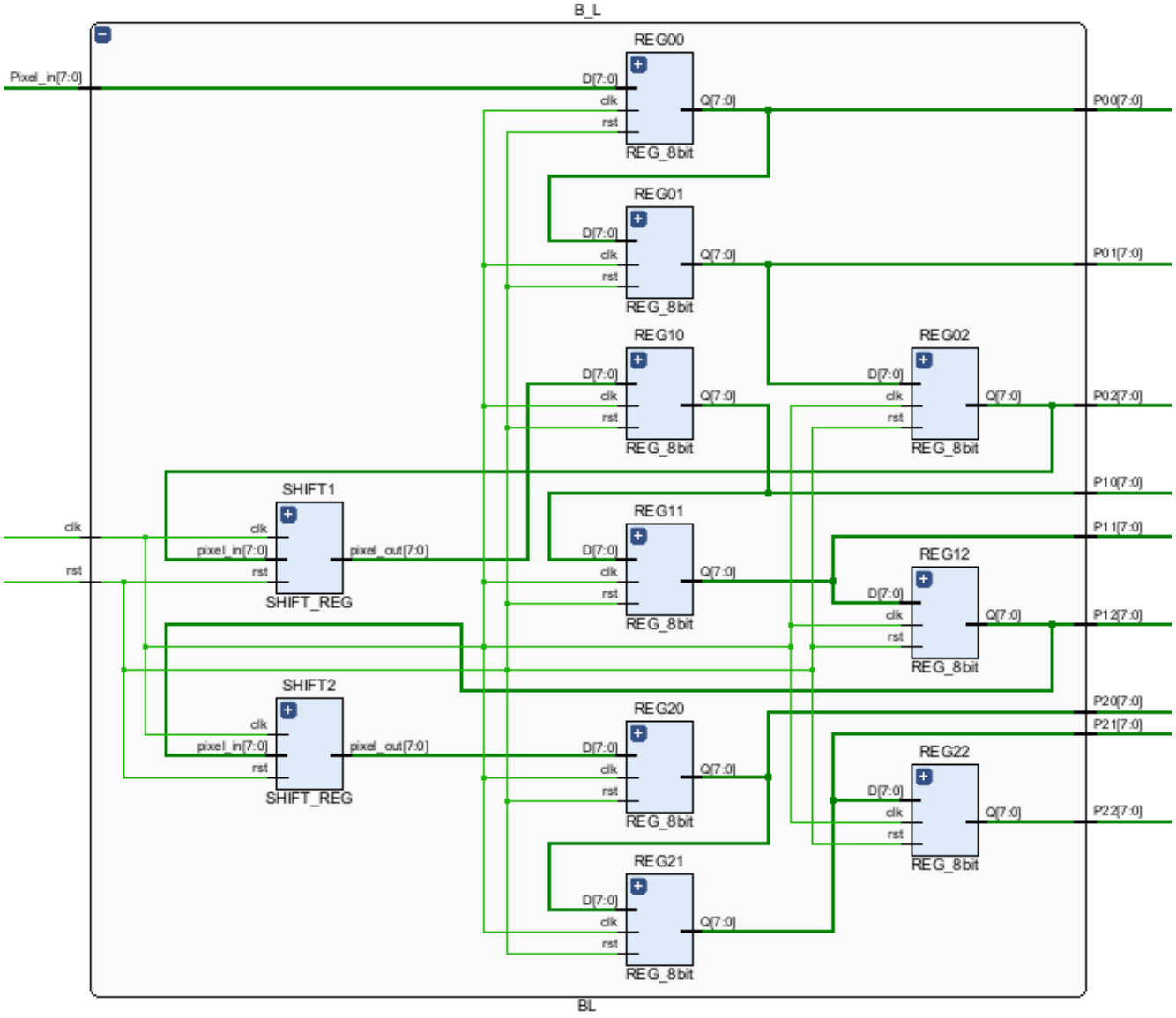
    process (clk, rst)
    begin
        if rst = '1' then
            Q <= (others => '0');
        elsif rising_edge(clk) then

                Q <= D;

        end if;
    end process;

end Behavioral;
```

Codice VHDL Registro a 8 bit



Schematic RTL Buffer Line

2.2 Convolutore

Questo blocco si occupa dell'elaborazione vera e propria: riceve i 9 pixel della finestra 3x3 dal blocco Buffer Line e i 3 coefficienti del filtro dal blocco FSM. Tramite i componenti interni, ovvero l'Adder Tree, il Moltiplicatore e il Carry Save a 3 ingressi, è in grado di produrre il pixel filtrato. La corretta pipeline, che mi permette di spezzare il path critico, è ottenuta mediante i registri istanziati nel process mediante i relativi signal.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;

entity CONV_ISO is
    port (
        clk, rst : in std_logic;
        Pixel_0, Pixel_1, Pixel_2, Pixel_3, Pixel_4, Pixel_5, Pixel_6, Pixel_7, Pixel_8 : in std_logic_vector(7 downto 0);
        W_in, WL_in, WC_in : in std_logic_vector(7 downto 0);
        Pixel_out : out std_logic_vector(19 downto 0)
    );
end CONV_ISO;

architecture RTL_level of CONV_ISO is

    component ADDER_TREE is
        port (
            A,B,C,D : in std_logic_vector(7 downto 0);
            sum_out : out std_logic_vector(9 downto 0)
        );
    end component;

    component MULTIPLIER is
        Port(
            MoltA: in std_logic_vector(9 downto 0);
            MoltB: in std_logic_vector(7 downto 0);
            Ris_Molt: out std_logic_vector(17 downto 0));
    end component;

    component CSA is
        port (
            Molt_W, Molt_WL, Molt_WC  : in std_logic_vector(17 downto 0);
            pixel_out : out std_logic_vector(19 downto 0)
        );
    end component;
```

```
    signal Pixel_center : std_logic_vector(7 downto 0);
    signal pixel_center_10bit : std_logic_vector(9 downto 0);

    signal Pixel_L_sum, Pixel_C_sum, Pixel_L_sum_reg, Pixel_C_sum_reg : std_logic_vector(9 downto 0);

    signal Pixel_L_int, Pixel_C_int,Pixel_CENTER_int : std_logic_vector(17 downto 0);

    signal center_reg, lateral_reg, corner_reg : std_logic_vector(17 downto 0);

    signal Pixel_out_int,Pixel_out_reg : std_logic_vector(19 downto 0);

begin

    ADDER_WL : ADDER_TREE port map(A => Pixel_1, B => Pixel_3, C => Pixel_5, D => Pixel_7, sum_out => Pixel_L_sum);
    ADDER_WC : ADDER_TREE port map(A => Pixel_0, B => Pixel_2, C => Pixel_6, D => Pixel_8, sum_out => Pixel_C_sum);

    pixel_center_10bit <= std_logic_vector(resize(unsigned(Pixel_center), 10));

    Mult_W  : MULTIPLIER port map(MoltA => pixel_center_10bit, MoltB => W_in,  Ris_Molt => Pixel_CENTER_int);
    Mult_WL : MULTIPLIER port map(MoltA => Pixel_L_sum_reg, MoltB => WL_in, Ris_Molt => Pixel_L_int);
    Mult_WC : MULTIPLIER port map(MoltA => Pixel_C_sum_reg, MoltB => WC_in, Ris_Molt => Pixel_C_int);

    Final_Adder : CSA port map( Molt_W => center_reg, Molt_WL => lateral_reg, Molt_WC => corner_reg, pixel_out => Pixel_out_int);


     process(clk, rst)

     begin

         if rst = '1' then

             Pixel_center    <= (others => '0');
             Pixel_L_sum_reg <= (others => '0');
             Pixel_C_sum_reg <= (others => '0');
             center_reg  <= (others => '0');
             lateral_reg <= (others => '0');
             corner_reg  <= (others => '0');

             Pixel_out_reg <= (others => '0');

         elsif rising_edge(clk) then

                 Pixel_center    <= Pixel_4;
                 Pixel_L_sum_reg <= Pixel_L_sum;
                 Pixel_C_sum_reg <= Pixel_C_sum;

                 center_reg  <= Pixel_CENTER_int;
                 lateral_reg <= Pixel_L_int;
                 corner_reg  <= Pixel_C_int;

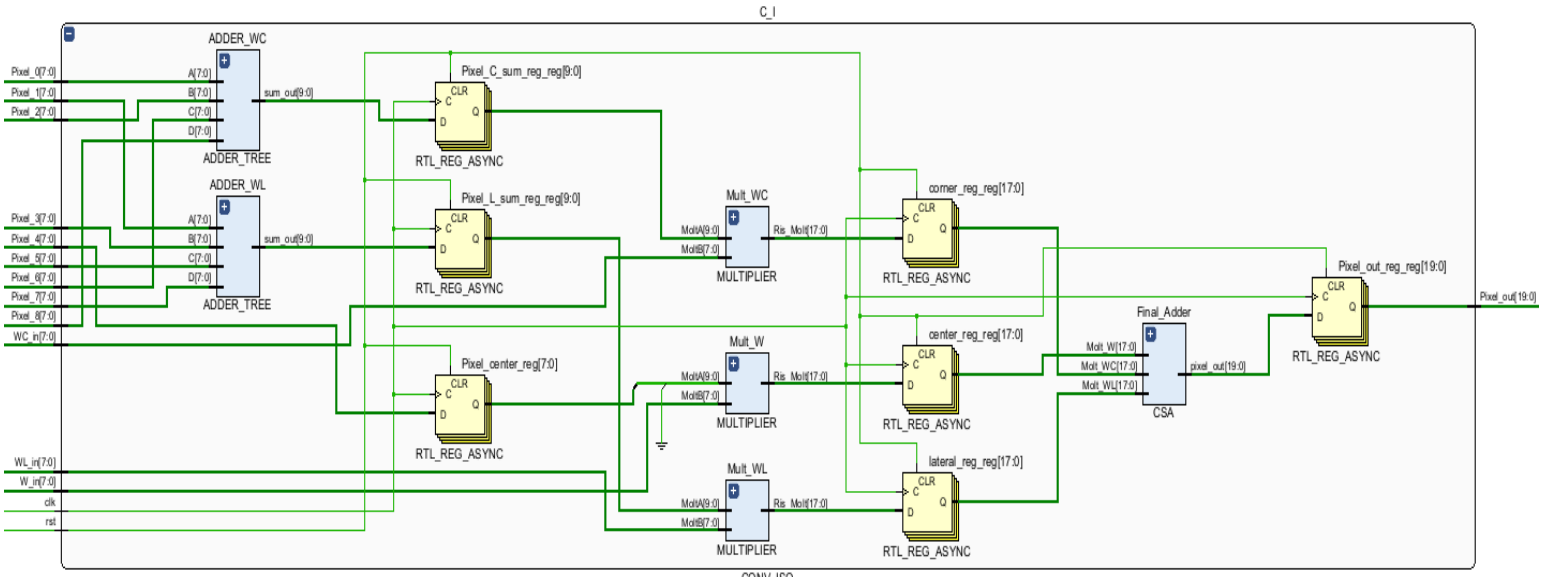
                 Pixel_out_reg <= Pixel_out_int;

         end if;
     end process;

     Pixel_out <= Pixel_out_reg;

end RTL level ;
```

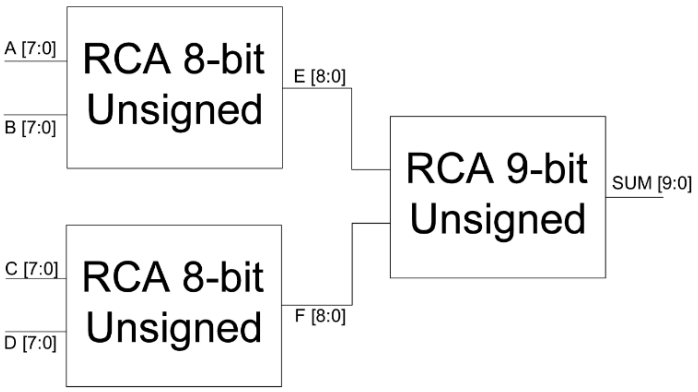
Codice VHDL Convolutore Isotropico



Schematic RTL Convolutore Isotropico

2.2.1 Adder Tree

Per eseguire la somma dei 4 pixel laterali e dei 4 pixel degli angoli, utilizzo 2 alberi di somma RCA based, come illustrato nella figura. I primi due RCA ricevono ingressi a 8 bit, mentre l'RCA che esegue la somma finale ha una dimensione di 9 bit. Il risultato finale è a 10 bit.



```vhdl
use IEEE.NUMERIC_STD.ALL;

entity ADDER_TREE is

    generic (N : integer := 8);

    port (A,B,C,D : in std_logic_vector(N-1 downto 0);
        sum_out    : out std_logic_vector(N+1 downto 0));
end ADDER_TREE;

architecture structural of ADDER_TREE is

    component RCA is
        generic (N : integer := 8);

        port (A_in,B_in    : in  std_logic_vector(N-1 downto 0);
            C_in    : in  std_logic;
            Sum     : out std_logic_vector(N-1 downto 0);
            C_out   : out std_logic);
    end component RCA;

    signal PS1,PS2 : std_logic_vector(N-1 downto 0);
    signal P_Cout_1,P_Cout_2,L2_Cout : std_logic;
    signal L2_PS1,L2_PS2,L2_SUM : std_logic_vector(N downto 0);

begin

    RCA0_L1 : RCA generic map (N => N)
        port map (A_in => A,B_in => B,C_in => '0', Sum => PS1,C_out => P_Cout_1);

    RCA1_L0 : RCA generic map (N => N)
        port map (A_in    => C,B_in => D,C_in => '0', Sum => PS2,C_out => P_Cout_2);

    L2_PS1 <= P_Cout_1 & PS1;
    L2_PS2 <= P_Cout_2 & PS2;

    RCA_L2 : RCA generic map (N => N + 1)
        port map (A_in => L2_PS1,B_in => L2_PS2,C_in => '0',Sum => L2_SUM,C_out => L2_Cout);

    sum_out <= L2_Cout & L2_SUM;

end architecture structural;
```

Codice VHDL Adder Tree

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA is
    port(
        A, B, Cin : in std_logic;
        S, Cout : out std_logic
    );
end FA;

architecture Behavioral of FA is

    signal p : std_logic;

begin
    p <= A xor B;
    S <= p xor Cin;
    Cout <= (A and B) or (Cin and p);

end Behavioral;
```

Codice VHDL FA

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA is

     generic (N : integer := 8);
    port (
        A_in,B_in : in  std_logic_vector(N-1 downto 0);
        C_in : in  std_logic;
        Sum : out std_logic_vector(N-1 downto 0);
        C_out : out std_logic
    );

end RCA;

architecture Behavioral of RCA is

component FA is

port(
    A,B,Cin:in std_logic;
    S,Cout:out std_logic);

end component FA;

signal C:std_logic_vector(N downto 0);

begin

C(0) <= C_in;

GenFA:for i in 0 to N-1 generate

MapFA: FA port map (A => A_in(i),B => B_in(i),Cin => C(i),S => Sum(i),Cout => C(i+1));
      end generate;

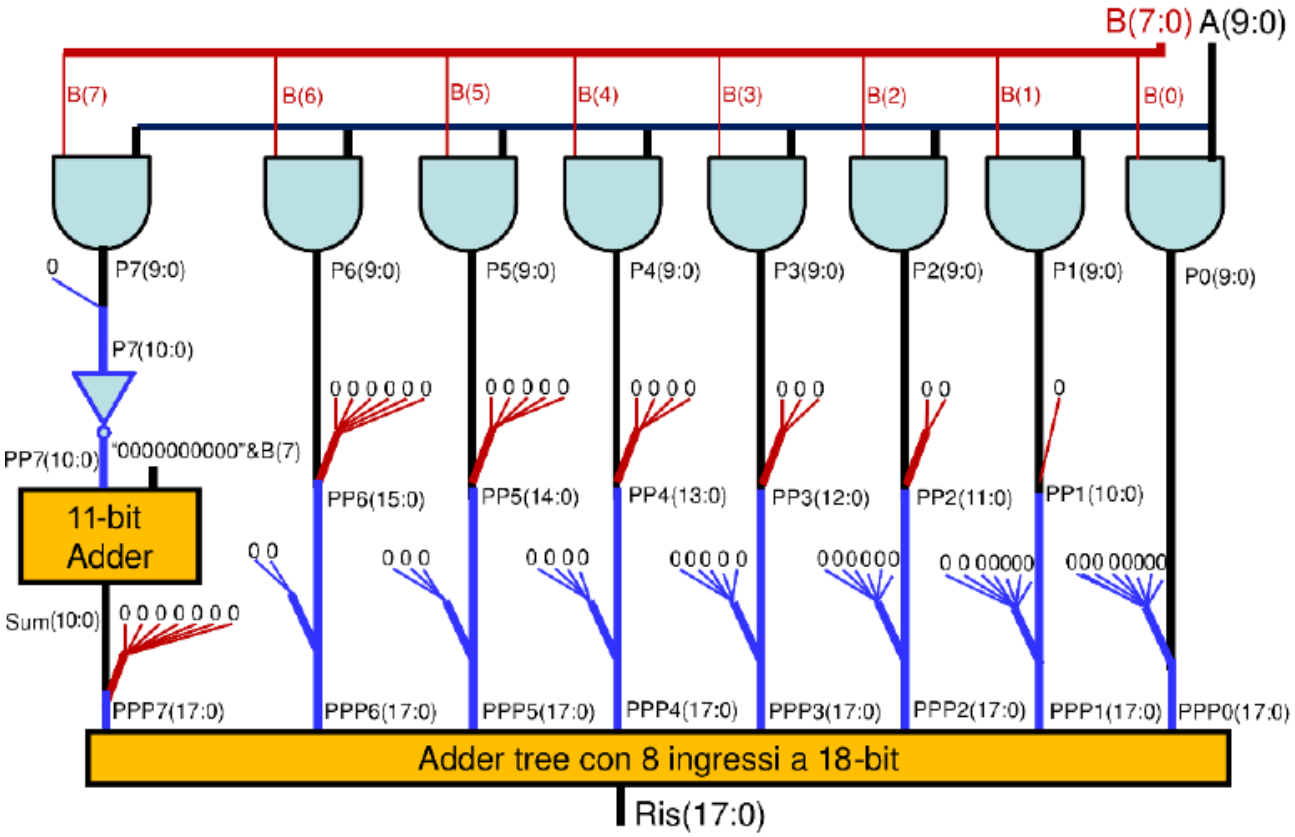
C_out <= C(N);

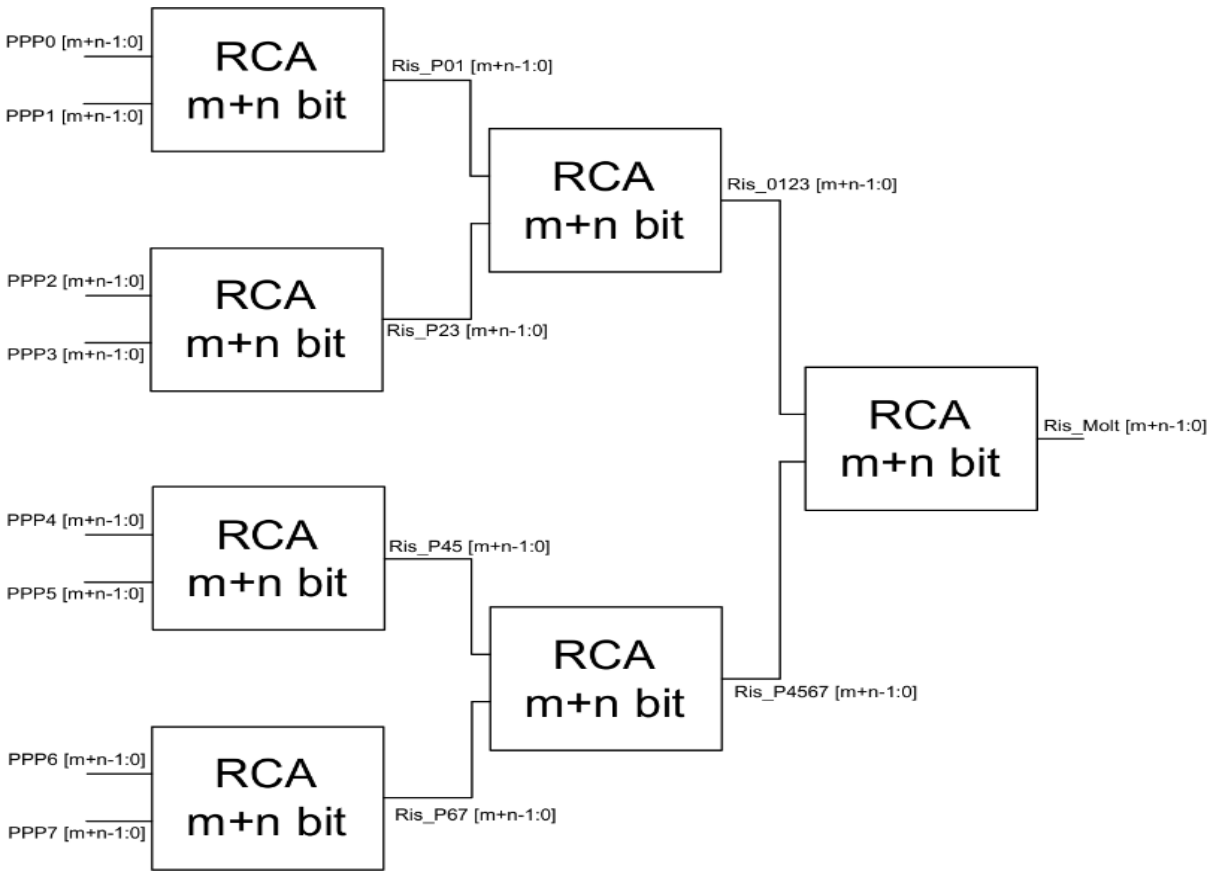

end Behavioral;
```

Codice VHDL RCA

2.2.2 Moltiplicatore

Il moltiplicatore progettato, di tipo “carta e penna”, esegue l'operazione tra un moltiplicando binario unsigned A a 10 bit (somma di 4 pixel laterali o corner) e un moltiplicatore binario signed B a 8 bit (coefficiente kernel). La procedura prevede la generazione di prodotti parziali ottenuti applicando l'AND bit a bit tra il moltiplicando A e il singolo bit del moltiplicatore B: se il bit di B è '1' si ricopia A, altrimenti il prodotto parziale è un vettore di zeri.
Questi prodotti parziali vengono successivamente shiftati a sinistra in base al peso del bit corrispondente del moltiplicatore. Le posizioni non definite dallo shift vengono riempite con zeri. Per sommare questi prodotti parziali, che hanno dimensioni diverse, è necessaria un'estensione del segno per uniformarli alla dimensione finale del risultato, pari a m+n (ovvero 18) bit. Tutti i prodotti parziali, eccetto l'ultimo (corrispondente al bit di segno di B), sono considerati unsigned e quindi estesi con zeri (zero padding). Per l'ultimo prodotto parziale, che tiene conto del segno del moltiplicatore B, si effettua prima un'estensione con uno 0 e successivamente il complemento a due, ottenuto invertendo tutti i bit e sommando 1.
Infine, dopo aver esteso tutti i prodotti parziali a 18 bit, essi vengono sommati utilizzando un albero di somma composto da sette RCA (lo stesso componente usato nell’Adder Tree). Per effettuare il prodotto tra il pixel centrale (8-bit) e il coefficiente del kernel (8-bit) in posizione omologa utilizzo il medesimo moltiplicatore estendendo la lunghezza del moltiplicando A a 10 bit con padding di zeri, eliminando la necessità di dover creare un altro moltiplicatore 8x8, ottimizzando in tal modo le risorse e la riusabilità dei componenti del circuito.




PPP0 [m+n-1:0]
PPP1 [m+n-1:0]
RCA
m+n bit
Ris_P01 [m+n-1:0]
PPP2 [m+n-1:0]
PPP3 [m+n-1:0]
RCA
m+n bit
Ris_P23 [m+n-1:0]
RCA
m+n bit
Ris_0123 [m+n-1:0]
PPP4 [m+n-1:0]
PPP5 [m+n-1:0]
RCA
m+n bit
Ris_P45 [m+n-1:0]
PPP6 [m+n-1:0]
PPP7 [m+n-1:0]
RCA
m+n bit
Ris_P67 [m+n-1:0]
RCA
m+n bit
Ris_P4567 [m+n-1:0]
RCA
m+n bit
Ris_Molt [m+n-1:0]

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MULTIPLIER is

generic(N: integer:=18);

Port
    (MoltA:in std_logic_vector(9 downto 0);
     MoltB:in std_logic_vector(7 downto 0);
     Ris_Molt:out std_logic_vector(N-1 downto 0)
     );

end MULTIPLIER;

architecture Behavioral of MULTIPLIER is

component RCA is

generic (N : integer := 8);
port (
    A_in,B_in  : in  std_logic_vector(N-1 downto 0);
    C_in : in  std_logic;
    Sum : out std_logic_vector(N-1 downto 0);
    C_out : out std_logic
);
end component;

signal PPP0,PPP1,PPP2,PPP3,PPP4,PPP5,PPP6: std_logic_vector(17 downto 0) := (others => '0');
signal Ris_P01,Ris_P23,Ris_P45,Ris_P67: std_logic_vector(18 downto 0);
signal Ris_P0123,Ris_P4567: std_logic_vector(19 downto 0);

signal P7_temp: std_logic_vector(17 downto 0) := (others => '0');
signal PPP7_inverted : std_logic_vector(17 downto 0);
signal PPP7_2s : std_logic_vector(17 downto 0);
signal PPP7_final : std_logic_vector(17 downto 0);

signal Ris_Molt_int : std_logic_vector(20 downto 0);
```

```
begin

Gen_PP : for i in 0 to 9 generate

    PPP0(i) <= MoltA(i) AND MoltB(0);
    PPP1(i+1) <= MoltA(i) AND MoltB(1);
    PPP2(i+2) <= MoltA(i) AND MoltB(2);
    PPP3(i+3) <= MoltA(i) AND MoltB(3);
    PPP4(i+4) <= MoltA(i) AND MoltB(4);
    PPP5(i+5) <= MoltA(i) AND MoltB(5);
    PPP6(i+6) <= MoltA(i) AND MoltB(6);
end generate ;

P7_temp(16 downto 7) <= MoltA;

PPP7_inverted <= not P7_temp;

Adder_2s: RCA generic map(18)

    port map(A_in => PPP7_inverted, B_in => (0 => '1', others => '0'), C_in  => '0', Sum  => PPP7_2s,C_out => open);

PP7: process(MoltB(7), PPP7_2s)
begin
    if MoltB(7) = '1' then
        PPP7_final <= PPP7_2s;
    else
        PPP7_final <= (others => '0');
    end if;
end process PP7;



RCA0_L0: RCA generic map(18)
    port map(A_in => PPP0, B_in => PPP1, C_in => '0', Sum => Ris_P01(17 downto 0), C_out => Ris_P01(18));
RCA1_L0: RCA generic map(18)
    port map(A_in => PPP2, B_in => PPP3, C_in => '0', Sum => Ris_P23(17 downto 0), C_out => Ris_P23(18));
RCA2_L0: RCA generic map(18)
    port map(A_in => PPP4, B_in => PPP5, C_in => '0', Sum => Ris_P45(17 downto 0), C_out => Ris_P45(18));
RCA3_L0: RCA generic map(18)
    port map(A_in => PPP6, B_in => PPP7_final, C_in => '0', Sum => Ris_P67(17 downto 0), C_out => Ris_P67(18));
RCA0_L1: RCA generic map(19)
    port map(A_in => Ris_P01, B_in => Ris_P23, C_in => '0', Sum => Ris_P0123(18 downto 0), C_out => Ris_P0123(19));
RCA1_L1: RCA generic map(19)
    port map(A_in => Ris_P45, B_in => Ris_P67, C_in => '0', Sum => Ris_P4567(18 downto 0), C_out => Ris_P4567(19));
RCA_L2: RCA generic map(20)
    port map(A_in => Ris_P0123, B_in => Ris_P4567, C_in => '0', Sum => Ris_Molt_int(19 downto 0), C_out => Ris_Molt_int(20));

Ris_Molt <= Ris_Molt_int(N-1 downto 0);

end Behavioral;
```

Codice VHDL Moltiplicatore

2.2.3 CSA

Questo blocco è progettato per sommare i risultati delle tre moltiplicazioni, ottenendo così il valore del pixel dell'immagine filtrata. Per ottimizzare il ritardo in questa fase finale, è stato scelto di utilizzare un sommatore Carry-Save che presenta un ritardo di τFA + τRCA(20). Il primo step per eseguire la somma di tre operandi utilizzando la logica Carry-Save prevede di sommare i bit corrispondenti di ogni posizione attraverso un Full-Adder per ciascuna posizione. Il Full-Adder riceve tre input (A, B e Cin) e produce due output: la somma (S) e il riporto (Cout). In questo modo si ottengono due vettori distinti: un vettore di somma parziale e un vettore dei riporti. Nel secondo step, i due vettori vengono allineati: il vettore dei riporti viene shiftato di una posizione verso sinistra, mentre il vettore delle somme parziali viene esteso per avere la stessa dimensione. Infine, si esegue la somma di questi due vettori utilizzando un RCA.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;

entity CSA is
    port(Molt_W, Molt_WL, Molt_WC : in std_logic_vector(17 downto 0);
        pixel_out : out std_logic_vector(19 downto 0));
end CSA;

architecture Behavioral of CSA is

    component FA is
        port(A, B, Cin : in std_logic;
            S, Cout : out std_logic);
    end component FA;

    component RCA is
        generic (N:integer:=8);
        port(A_in, B_in : in std_logic_vector(N-1 downto 0);
            C_in : in std_logic;
            Sum : out std_logic_vector(N-1 downto 0);
            C_out : out std_logic);
    end component;

    signal Csa_Sum, Csa_Carry : std_logic_vector(17 downto 0);
    signal Molt_W_signed, Molt_WL_signed, Molt_WC_signed : signed(17 downto 0);
    signal Csa_Sum_signed, Csa_Carry_signed : signed(17 downto 0);

    signal RCA_A_signed, RCA_B_signed, RCA_sum_signed : signed (19 downto 0);
    signal Final_sum : std_logic_vector (19 downto 0);
```

```
begin

    Molt_WC_signed<=signed(Molt_WC);
    Molt_WL_signed<=signed(Molt_WL);
    Molt_W_signed<=signed(Molt_W);

    Stage1 : for i in 0 to 17 generate
        FA_i: FA port map (A=>Molt_WC_signed(i),B=>Molt_WL_signed(i),Cin=>Molt_W_signed(i), S=>Csa_Sum(i),Cout=>Csa_Carry(i));
    end generate;

    Csa_Sum_signed<=signed(Csa_Sum);
    Csa_Carry_signed<=signed(Csa_Carry);

    RCA_A_signed<=resize(Csa_Sum_signed,20);
    RCA_B_signed<=resize(Csa_Carry_signed & '0',20);

    Final_Adder: RCA generic map(N=>20)

    port map(A_in=>std_logic_vector(RCA_A_signed), B_in=>std_logic_vector(RCA_B_signed), C_in=>'0', Sum=>Final_sum, C_out=>open);

    RCA_sum_signed<=signed(Final_sum);
    pixel_out<=std_logic_vector(RCA_sum_signed);

end Behavioral;
```

Codice VHDL CSA

2.3 Finite State Machine

La macchina a stati finiti gestisce l'elaborazione dell'immagine ed è composta da cinque stati:

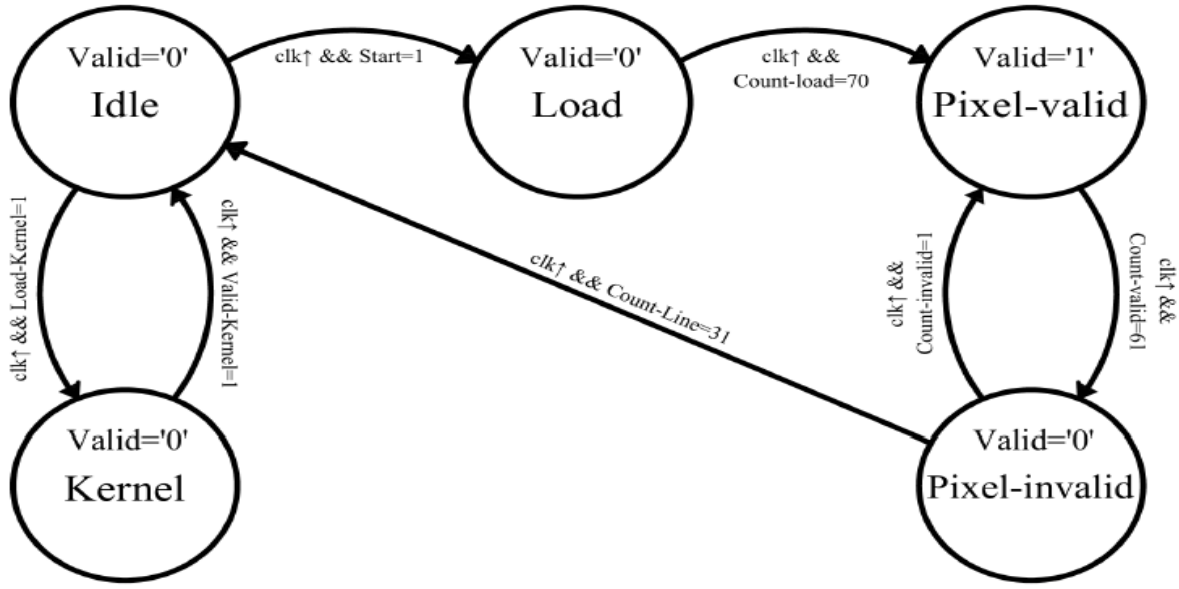
- Idle: Stato di inattività in cui il sistema attende un segnale di Start per avviare l'elaborazione o un segnale di Load-Kernel per caricare i coefficienti del filtro. In questo stato, il segnale Valid è basso.

- Kernel: Stato in cui vengono memorizzati i coefficienti del filtro. Una volta completato il caricamento, il sistema torna allo stato di Idle. Anche in questo stato, il segnale Valid è basso.

- Load: Stato attivato dal segnale Start, in cui si rimane fino a quando la buffer line non è completamente riempita dai pixel e non sono trascorsi eventuali cicli di clock aggiuntivi dovuti agli stadi di pipeline implementati. Il passaggio allo stato successivo, Pixel-Valid, avviene quando il contatore che traccia i cicli di clock raggiunge il valore 132. Anche in questo stato, il segnale Valid è basso.

- Pixel-Valid: Stato in cui i pixel prodotti sono considerati validi. Questo stato dura per 62 cicli di clock, durante i quali vengono generati 62 pixel validi. Al termine di questo intervallo, si passa allo stato Pixel-Invalid. In questo stato, il segnale Valid è alto.

- Pixel-Invalid: Stato in cui i pixel prodotti non sono validi, poiché si riferiscono alle finestre costruite sui bordi dell'immagine. Il sistema rimane in questo stato per 2 cicli di clock. Ogni volta che la FSM arriva in questo stato, si incrementa un contatore per tenere traccia del numero di righe: quando il contatore raggiunge 62 righe, la FSM torna allo stato Idle. Anche in questo stato, il segnale Valid è basso. Come si può vedere dall'entity del blocco FSM, i coefficienti del filtro WC, WL e W sono definiti come segnali di output del blocco. Una volta memorizzati nei registri, i valori presenti sui segnali LoadWC, LoadWL e LoadW vengono forniti al blocco che effettua la convoluzione, tramite il blocco di controllo.




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity FSM is
    Port(
        clk, rst, Start, Load_Kernel, Valid_Kernel : in std_logic;
        Load_W, Load_WL, Load_WC : in std_logic_vector(7 downto 0);
        W, WL, WC : out std_logic_vector(7 downto 0);
        Valid : out std_logic
    );
end FSM;

architecture Behavioral of FSM is

    type state_type is (idle, kernel, load, pixel_valid, pixel_invalid);
    signal state : state_type;

    signal count_load : std_logic_vector(7 downto 0) := (others => '0');
    signal count_pixel_valid : std_logic_vector(5 downto 0) := (others => '0');
    signal count_pixel_invalid : std_logic_vector(2 downto 0) := (others => '0');
    signal count_line : std_logic_vector(6 downto 0) := (others => '0');

begin

    process(clk)

    begin
        if rising_edge(clk) then
            if rst = '1' then
                state <= idle;
                count_load <= (others => '0');
                count_pixel_valid <= (others => '0');
                count_pixel_invalid <= (others => '0');
                count_line <= (others => '0');
                W  <= (others => '0');
                WL <= (others => '0');
                WC <= (others => '0');
```

```
            else
                case state is

                    when idle =>
                        if Start = '1' then
                            state <= load;
                        elsif Load_Kernel = '1' then
                            state <= kernel;
                        else
                            state <= idle;
                        end if;

                    when kernel =>
                        if Valid_Kernel = '1' then

                            W  <= Load_W;
                            WL <= Load_WL;
                            WC <= Load_WC;

                            state <= idle;
                        else
                            state <= kernel;
                        end if;

                    when load =>

                        if count_load = "10000101" then
                            state <= pixel_valid;
                            count_load <= (others => '0');
                        else
                            state <= load;
                            count_load <= count_load + 1;
                        end if;

                    when pixel_valid =>

                        if count_pixel_valid = "111101" then
                            state <= pixel_invalid;
                            count_pixel_valid <= (others => '0');
                            count_line <= count_line + 1;
                        else
                            state <= pixel_valid;
                            count_pixel_valid <= count_pixel_valid + 1;
                        end if;

                    when pixel_invalid =>
                        if count_pixel_invalid = "001" then
                            count_pixel_invalid <= (others => '0');
                            state <= pixel_valid;
                        elsif count_line = "0111110" then
                            state <= idle;
                            count_line <= (others => '0');
                        else
                            state <= pixel_invalid;
                            count_pixel_invalid <= count_pixel_invalid + 1;
                        end if;

                end case;
            end if;
        end if;
    end process;

    process(state)
    begin
        case state is

                when idle => Valid <= '0';
                when kernel => Valid <= '0';
                when load => Valid <= '0';
                when pixel_valid => Valid <= '1';
                when pixel_invalid => Valid <= '0';

        end case;
    end process;

end Behavioral;
```

Codice VHDL FSM

2.4 Top Design

Una volta descritti i tre componenti principali per realizzare il circuito ideato, ovvero buffer line, convolutore e FSM, questi vengono collegati opportunamente. Il circuito finale presenta l'uscita del pixel filtrato (Pixel_out), rappresentato in 20 bit.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TOP_ELABORATION is

    Port(
        clk, rst, Start, Load_Kernel, Valid_Kernel : in std_logic;
        Load_WC, Load_WL, Load_W : in std_logic_vector(7 downto 0);
        Pixel_in : in std_logic_vector(7 downto 0);
        Pixel_out : out std_logic_vector(19 downto 0);
        Valid : out std_logic
    );
end TOP_ELABORATION;

architecture Behavioral of TOP_ELABORATION is

    signal WC_int, WL_int, W_int : std_logic_vector(7 downto 0);

    signal P00_int, P01_int, P02_int, P10_int, P11_int, P12_int, P20_int, P21_int, P22_int : std_logic_vector(7 downto 0);

    component FSM is
        Port(
            clk, rst, Start, Load_Kernel, Valid_Kernel : in std_logic;
            Load_W, Load_WL, Load_WC : in std_logic_vector(7 downto 0);
            W, WL, WC : out std_logic_vector(7 downto 0);
            Valid : out std_logic
        );
    end component;

    component BL is
        Port(
            clk, rst: in std_logic;
            Pixel_in : in std_logic_vector(7 downto 0);
            P00, P01, P02, P10, P11, P12, P20, P21, P22 : out std_logic_vector(7 downto 0)
        );
    end component;
```

```
    component CONV_ISO is
        port (
            clk, rst : in std_logic;
            Pixel_0, Pixel_1, Pixel_2, Pixel_3, Pixel_4, Pixel_5, Pixel_6, Pixel_7, Pixel_8 : in std_logic_vector(7 downto 0);
            W_in, WL_in, WC_in : in std_logic_vector(7 downto 0);
            Pixel_out : out std_logic_vector(19 downto 0)
        );
    end component;

begin

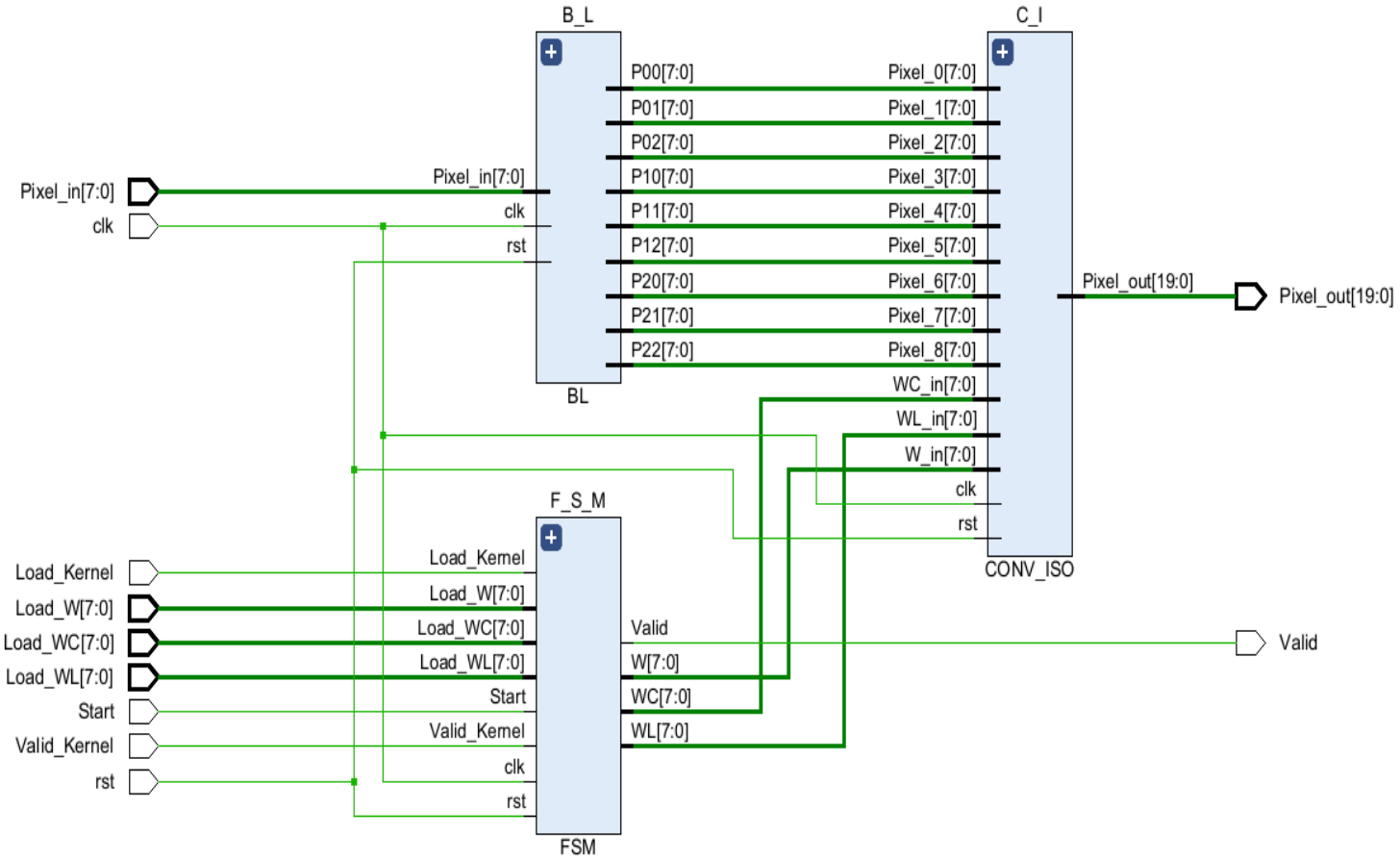
    F_S_M : FSM
        port map(
            clk => clk, rst => rst, Start => Start, Load_Kernel => Load_Kernel, Valid_Kernel => Valid_Kernel,
            Load_W => Load_W, Load_WL => Load_WL, Load_WC => Load_WC,
            W => W_int, WL => WL_int, WC => WC_int,
            Valid => Valid
        );

    B_L : BL
        port map(
            Pixel_in => Pixel_in, clk => clk, rst => rst,
            P00 => P00_int, P01 => P01_int, P02 => P02_int,
            P10 => P10_int, P11 => P11_int, P12 => P12_int,
            P20 => P20_int, P21 => P21_int, P22 => P22_int
        );

    C_I : CONV_ISO
        port map(
            clk => clk, rst => rst,
            Pixel_0 => P00_int, Pixel_1 => P01_int, Pixel_2 => P02_int,
            Pixel_3 => P10_int, Pixel_4 => P11_int, Pixel_5 => P12_int,
            Pixel_6 => P20_int, Pixel_7 => P21_int, Pixel_8 => P22_int,
            W_in => W_int, WL_in => WL_int,WC_in => WC_int, Pixel_out => Pixel_out
        );

end Behavioral;
```

Codice VHDL Top Elaboration



Schematic RTL Top Elaboration

3 Simulazione

In prima istanza testo il moltiplicatore creando un opportuno testbench che mi scrive su file di testo tutte le possibili moltiplicazioni tra A unsigned 10 bit e B signed 8 bit riportando, per brevità, solo alcune delle prime e ultime operazioni, che confermano la validità del moltiplicatore.

```
  2 |   124 |        248
  2 |   125 |        250
  2 |   126 |        252
  2 |   127 |        254
  2 |  -128 |       -256
  2 |  -127 |       -254
  2 |  -126 |       -252
  2 |  -125 |       -250
```

```
1023 |    -7 |      -7161
1023 |    -6 |      -6138
1023 |    -5 |      -5115
1023 |    -4 |      -4092
1023 |    -3 |      -3069
1023 |    -2 |      -2046
1023 |    -1 |      -1023
```

Risulta evidente che il moltiplicatore esegue correttamente i calcoli per qualsiasi input.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;

entity TB_MULTIPLIER is
end TB_MULTIPLIER;

architecture Behavioral of TB_MULTIPLIER is

component MULTIPLIER is
generic(N: integer:=18);
Port(MoltA:in std_logic_vector(9 downto 0);
     MoltB:in std_logic_vector(7 downto 0);
     Ris_Molt:out std_logic_vector(N-1 downto 0));
end component;

signal TB_MoltA: std_logic_vector(9 downto 0) := (others => '0');
signal TB_MoltB: std_logic_vector(7 downto 0) := (others => '0');
signal TB_Ris_Molt: std_logic_vector(17 downto 0) := (others => '0');

begin

uut: MULTIPLIER
generic map (N => 18)
port map (MoltA => TB_MoltA, MoltB => TB_MoltB, Ris_Molt => TB_Ris_Molt
);


process
    variable wrline:line;
    file result_file:text open write_mode is "C:\Users\Danil\desktop\progetto\Molt.txt";

    begin
    wait for 104 ns;

    write(wrline, string'("MoltA_int | MoltB_int | Ris_Molt_int"));
    writeline(result_file, wrline);
    write(wrline, string'("----------|-----------|--------------"));
    writeline(result_file, wrline);

    for i in 0 to 1023 loop
        for j in 0 to 255 loop
            TB_MoltA <= std_logic_vector(to_unsigned(i,10));
            TB_MoltB <= std_logic_vector(to_signed(j,8));

            wait for 5 ns;

            write(wrline, to_integer(unsigned(TB_MoltA)), right, 5);
            write(wrline, string'(" | "));
            write(wrline, to_integer(signed(TB_MoltB)), right, 5);
            write(wrline, string'(" | "));
            write(wrline, to_integer(signed(TB_Ris_Molt)), right, 10);
            writeline(result_file, wrline);
        end loop;
    end loop;

    wait;
end process;

end Behavioral;
```

*Codice VHDL **TB** Moltiplicatore*

Successivamente creo un testbench per l'intera struttura per verificarne il corretto funzionamento, impostando i coefficienti del filtro W, WL, WC in maniera tale da realizzare un Laplaciano per Edge Detection.

0	-1	0
-1	4	-1
0	-1	0

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;
use STD.TEXTIO.ALL;


entity TB_TOP_ELABORATION is
end TB_TOP_ELABORATION;

architecture Behavioral of TB_TOP_ELABORATION is

component TOP_ELABORATION is

  Port(
        clk, rst, Start, Load_Kernel, Valid_Kernel : in std_logic;
        Load_WC, Load_WL, Load_W : in std_logic_vector(7 downto 0);
        Pixel_in : in std_logic_vector(7 downto 0);
        Pixel_out : out std_logic_vector(19 downto 0);
        Valid : out std_logic
    );
end component;

signal TB_clk,TB_rst,TB_Start,TB_Load_Kernel,TB_Valid_Kernel : std_logic:='0';
signal TB_Load_WC,TB_Load_WL,TB_Load_W,TB_Pixel_in: std_logic_vector(7 downto 0):=(others => '0');
signal TB_Valid: std_logic;
signal TB_Pixel_out: std_logic_vector(19 downto 0);
constant clk_period : time:=6ns;

begin

uut: TOP_ELABORATION
            port map ( clk => TB_clk, rst => TB_rst, Start => TB_Start,
            Load_Kernel => TB_Load_Kernel, Valid_Kernel   => TB_Valid_Kernel,
            Load_W => TB_Load_W, Load_WL => TB_Load_WL, Load_WC => TB_Load_WC,
            Pixel_in => TB_Pixel_in, Pixel_out=> TB_Pixel_out,
            Valid => TB_Valid
            );
```

```
process
    begin
        wait for clk_period/2;
        TB_clk<=not(TB_clk);
end process;

process
        variable rdline : line;
        variable tmp : integer;
        file Read_file : text open read_mode is "C:\\Users\\Danil\\Desktop\\lena64.txt";
        begin
        wait for 100*clk_period;
        TB_Load_Kernel<='1';
        wait for 2*clk_period;
        TB_Load_Kernel<='0';
        TB_Valid_Kernel<='1';

        TB_Load_WC<="00000000"; --0
        TB_Load_WL<="11111111"; -- -1
        TB_Load_W<="00000100"; -- 4

        wait for 2*clk_period;
        TB_Valid_Kernel<='0';
        TB_Load_WC<=(others=>'0');
        TB_Load_WL<=(others=>'0');
        TB_Load_W<=(others=>'0');
        wait for 5*clk_period;
        TB_Start<='1';
        wait for clk_period;
        TB_Start<='0';

        while not endfile(Read_file) loop
                readline(Read_file, rdline);
                read(rdline, tmp);
                TB_Pixel_in<=std_logic_vector(to_unsigned(tmp,8));
            wait for clk_period;
            end loop;
         wait;
end process;


process
        variable wrline:line;
        file result_file:text open write_mode is "C:\\Users\\Danil\\Desktop\\ElabXX.txt";
        begin
        if(TB_Valid='1') then

             write(wrline,TB_Pixel_out(10 downto 0),right);
             writeline(result_file,wrline);
      end if;
      wait for clk_period;
end process;


end Behavioral;
```

*Codice VHDL **TB** Top Elaboration*

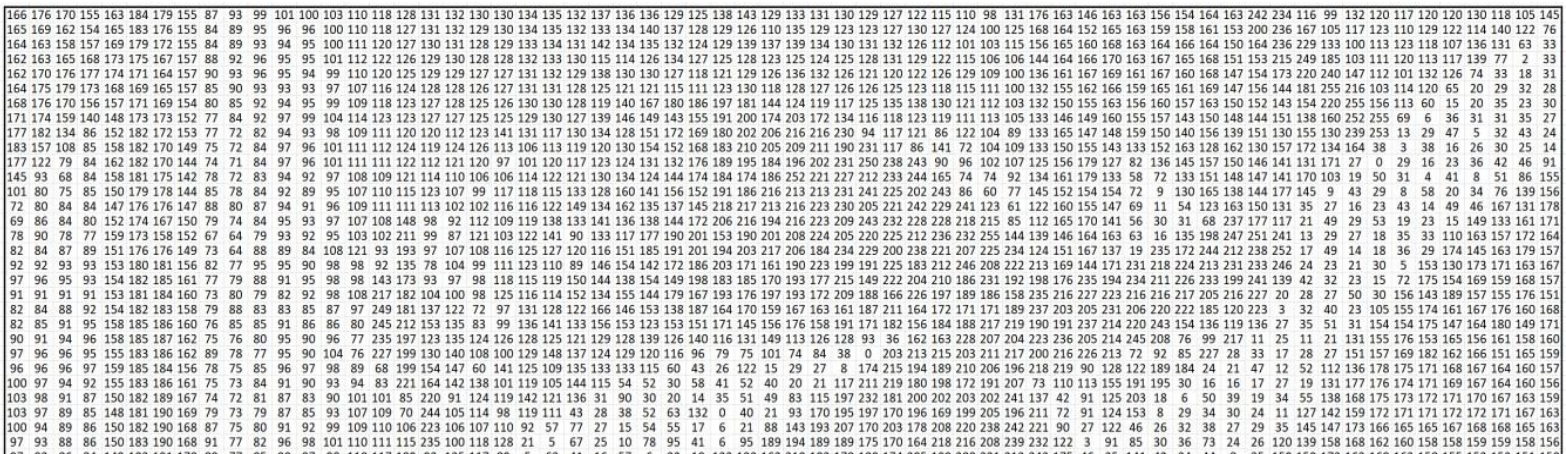
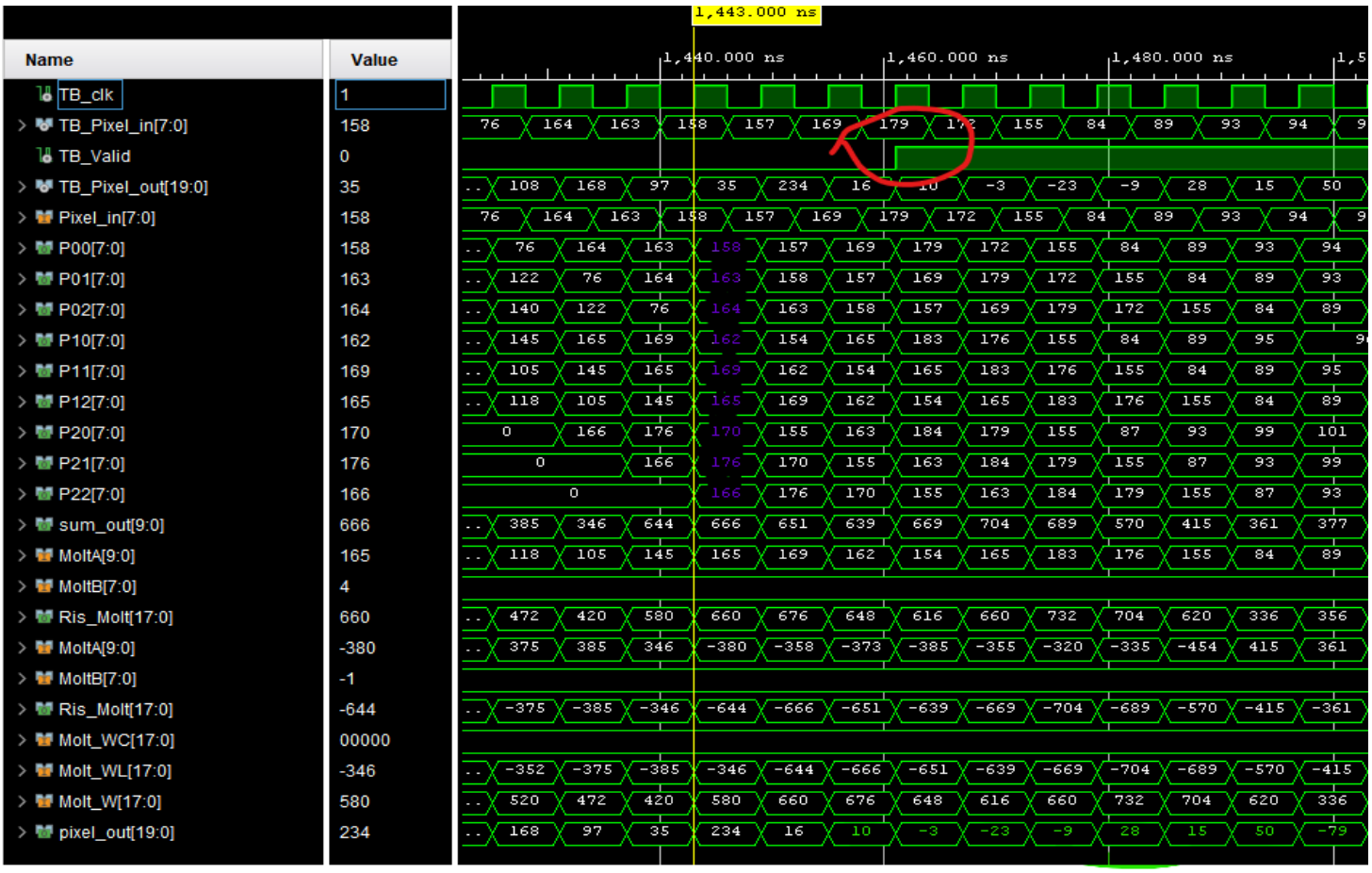
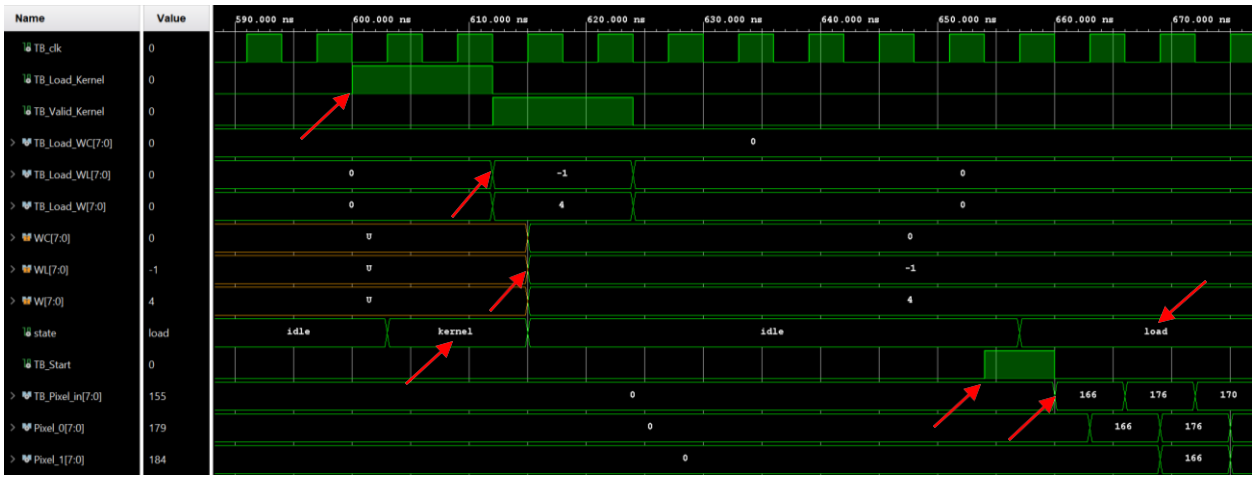


Immagine di partenza (Matrice 64X64)

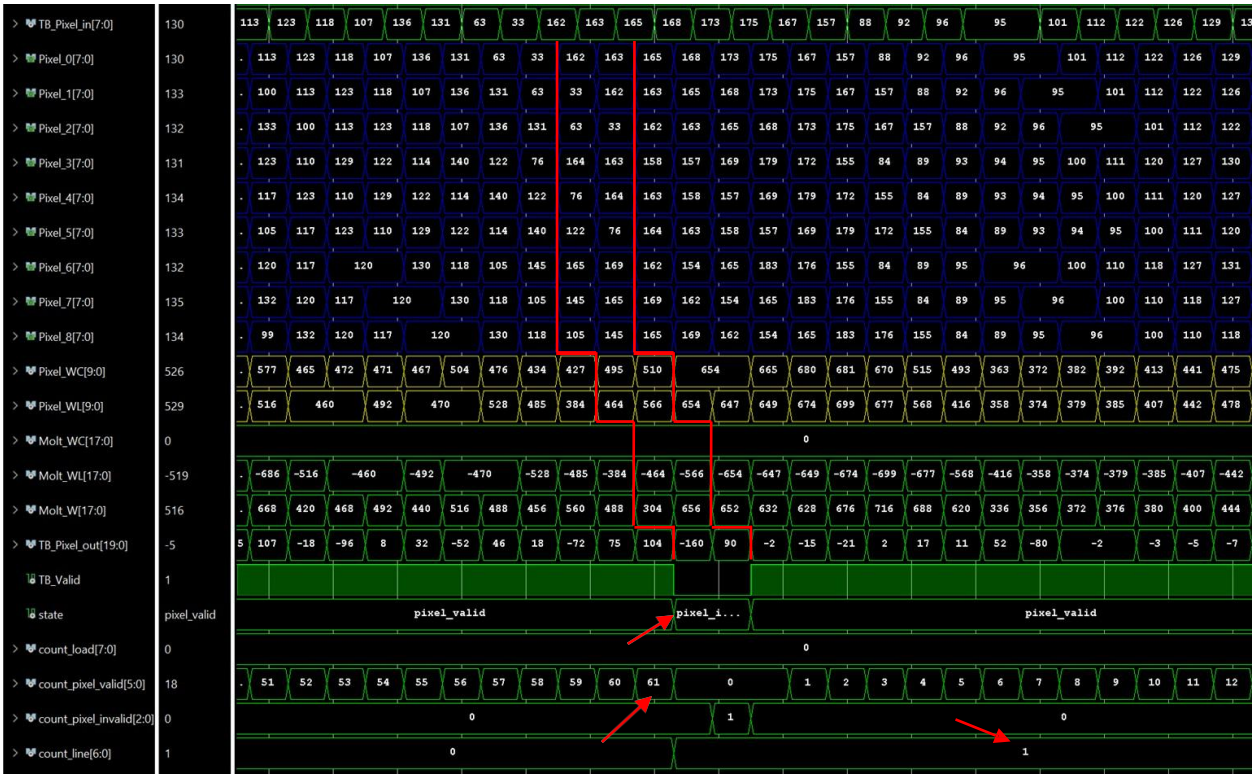


Da queste waveform si può chiaramente notare come la buffer line abbia generato in output parallelo i primi 9 pixel (evidenziati) necessari alla creazione della prima finestra e che, ad ogni successivo colpo di clock la finestra si sposta esattamente come previsto sull'immagine. 3 colpi di clock dopo (pipeline convolutore) possiamo apprezzare il primo risultato valido (10) corrispondente al primo pixel filtrato e così via. Quando la FSM passa allo stato Pixel_Valid, il contatore Count_pixel_valid inizia a incrementare per registrare il numero di pixel validi elaborati e si alza il segnale Valid.

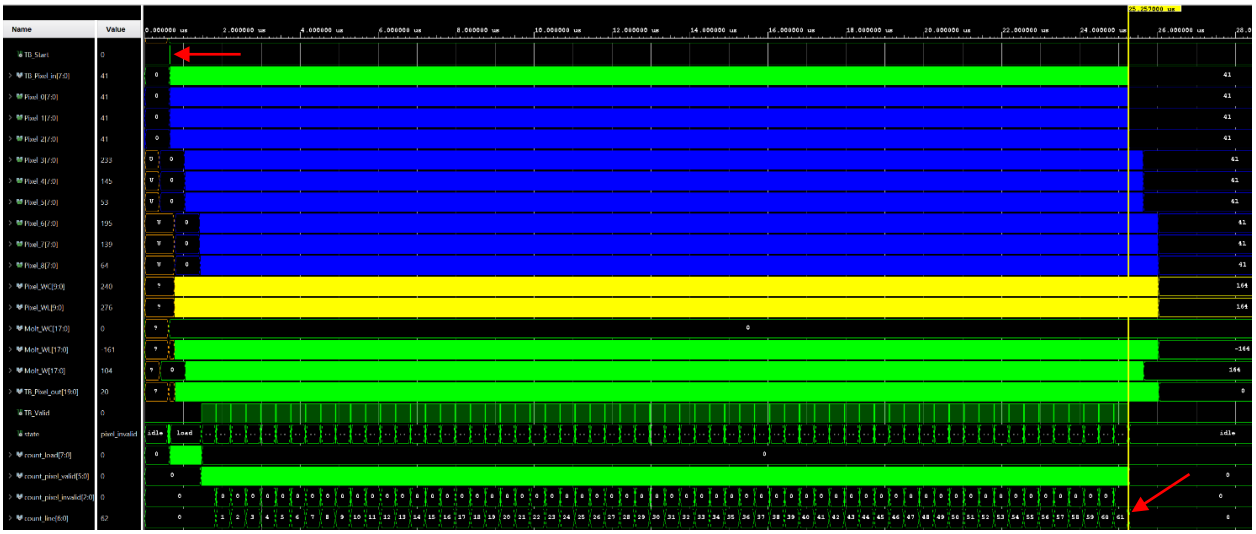
| | | |



Dalla Waveform generata da Vivado, si può osservare che, inizialmente, viene attivato il segnale Load_Kernel, permettendo alla FSM di passare allo stato Kernel. Successivamente, i coefficienti del filtro vengono inviati sugli I/O, accompagnati dal segnale Valid_Kernel. Al colpo di clock successivo, questi valori vengono memorizzati ed è possibile osservare l'uscita dei registri contenente i coefficienti corretti. Infine, dopo un determinato periodo di tempo, viene inviato un segnale di Start, consentendo alla FSM di passare allo stato Load. Al colpo di clock successivo, inizia la lettura del file "lena64.txt".



Dalla Waveform generata da Vivado, si può notare che, quando Count_pixel_valid raggiunge il valore 61, la FSM passa allo stato Pixel_Invalid, disattiva il segnale Valid poiché questi due pixel in uscita non devono essere considerati, e incrementa il contatore Count_Line, che tiene traccia del numero di righe dell'immagine. Dopo due cicli di clock, la FSM ritorna nello stato Pixel_Valid e si riattiva il segnale Valid.



Dalla Waveform generata da Vivado si può notare che, una volta che il contatore delle righe raggiunge 61, la FSM ritorna allo stato Idle, segnando la conclusione dell'elaborazione dell'intera immagine.

E’ stato sviluppato un codice in linguaggio C, progettato per simulare il comportamento del convolutore isotropico a livello software. Si utilizza l’immagine “Lena” in scala di grigi e di dimensioni 64x64 pixel, applicando il filtro isotropico laplaciano. Questo filtro è impiegato per il rilevamento dei contorni, ovvero evidenzia le variazioni di intensità nell'immagine. Il valore atteso generato da questa simulazione software è stato utilizzato come riferimento per confrontare i risultati ottenuti durante le simulazioni Behavioral e Post-Implementation su Vivado.

```c
#include <stdio.h>
#include <stdlib.h>
#define DIM 64

int main() {
    int Lena[DIM][DIM];
    FILE *file1 = fopen("lena64.txt", "r");

    for (int i=0; i<DIM; i++) {
        for (int j=0; j<DIM; j++) {
            fscanf(file1, "%d", &Lena[i][j]);
        }
    }
    fclose(file1);

    int Image_Processed[DIM][DIM];
    for (int i=1; i<DIM-1; i++) {
        for (int j= 1; j<DIM-1; j++) {
        Image_Processed[i][j]=4*Lena[i][j]-Lena[i-1][j]-Lena[i+1][j]-Lena[i][j-1]-Lena[i][j+1];
        }
    }

    FILE *file2 = fopen("Image_Processed.txt", "w");
    for (int i=1; i<DIM-1; i++) {
        for (int j=1; j<DIM-1; j++) {
        fprintf(file2, "%d\n", Image_Processed[i][j]);
        }
    }
    fclose(file2);
    return 0;
}
```



4 Post-Processing

L'immagine ottenuta da questa elaborazione ha una dimensione di 62x62 pixel, in cui ogni pixel è rappresentato in 20 bit signed. Di conseguenza, possono esserci valori negativi o superiori a 255, quindi al di fuori del range 0-255, rendendo impossibile la visualizzazione dell'immagine in scala di grigi a 8 bit. Per risolvere questo problema, è possibile utilizzare vari metodi per ottenere un'immagine con i pixel rappresentati a 8 bit. Il primo di questi metodi è la saturazione, che consiste nell'adeguare i valori dei pixel all'interno del range consentito. In pratica, se un pixel ha un valore maggiore di 255, viene "saturato" a 255, mentre se il suo valore è inferiore a 0, viene impostato a 0. Questo processo permette di garantire che tutti i pixel siano compresi nell'intervallo 0-255, rendendo così l'immagine visualizzabile in scala di grigi a 8 bit. Questa operazione viene svolta su un foglio di calcolo Excel.



Immagine ottenuta dal filtraggio
con Saturazione a 8 bit

Un altro metodo utile per ridurre il numero di bit da 20 a 8 è la normalizzazione lineare. Questo processo consente di ridimensionare i valori dei pixel in modo che rientrino nel range 0-255. Per applicare questa tecnica, è necessario prima determinare il valore massimo e il valore minimo dei pixel nell'immagine. Una volta ottenuti questi valori, si applica la seguente formula a ciascun pixel:
Pixel − min(Pixel) New_Pixel = 255 × max(Pixel) − min(Pixel). Questa formula permette di scalare i valori dei pixel in modo che il valore minimo venga trasformato in 0 e quello massimo in 255, adattando così l'immagine per una visualizzazione corretta in scala di grigi a 8 bit. Anche questo processo viene eseguito su un foglio di calcolo Excel.



Immagine ottenuta dal filtraggio con Normalizzazione Lineare a 8 bit

Un’ altra simulazione è stata effettuata modificando il test bench per applicare un filtro 3x3 utilizzando i coefficienti interi W=4 (centro), WL=2 (laterali) e WC=1 (angolari). Questi valori corrispondono a un tipico kernel Gaussiano che, una volta normalizzato (dividendo per la somma dei coefficienti, 16 in questo caso), agisce come un filtro di media pesata. Il risultato è, come previsto, un'immagine sfocata ma pienamente riconoscibile, con bordi ammorbiditi e riduzione del rumore, poiché il Gaussiano preserva la struttura complessiva e produce valori positivi.

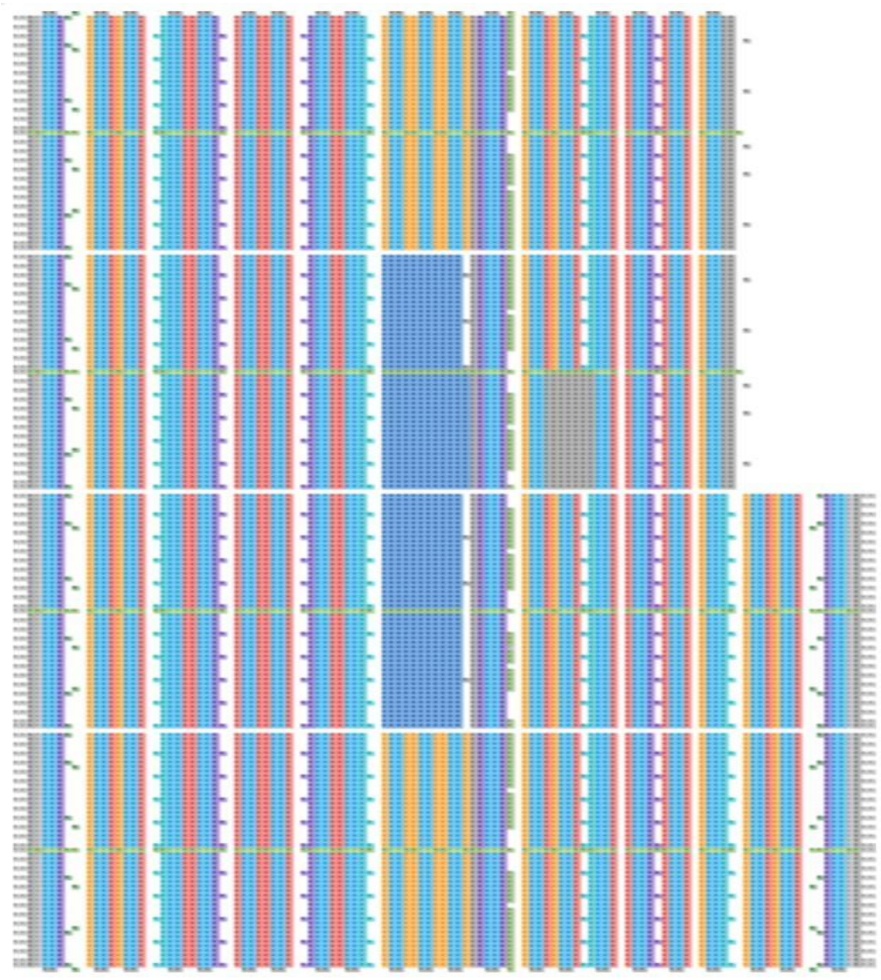


5 Implementazione

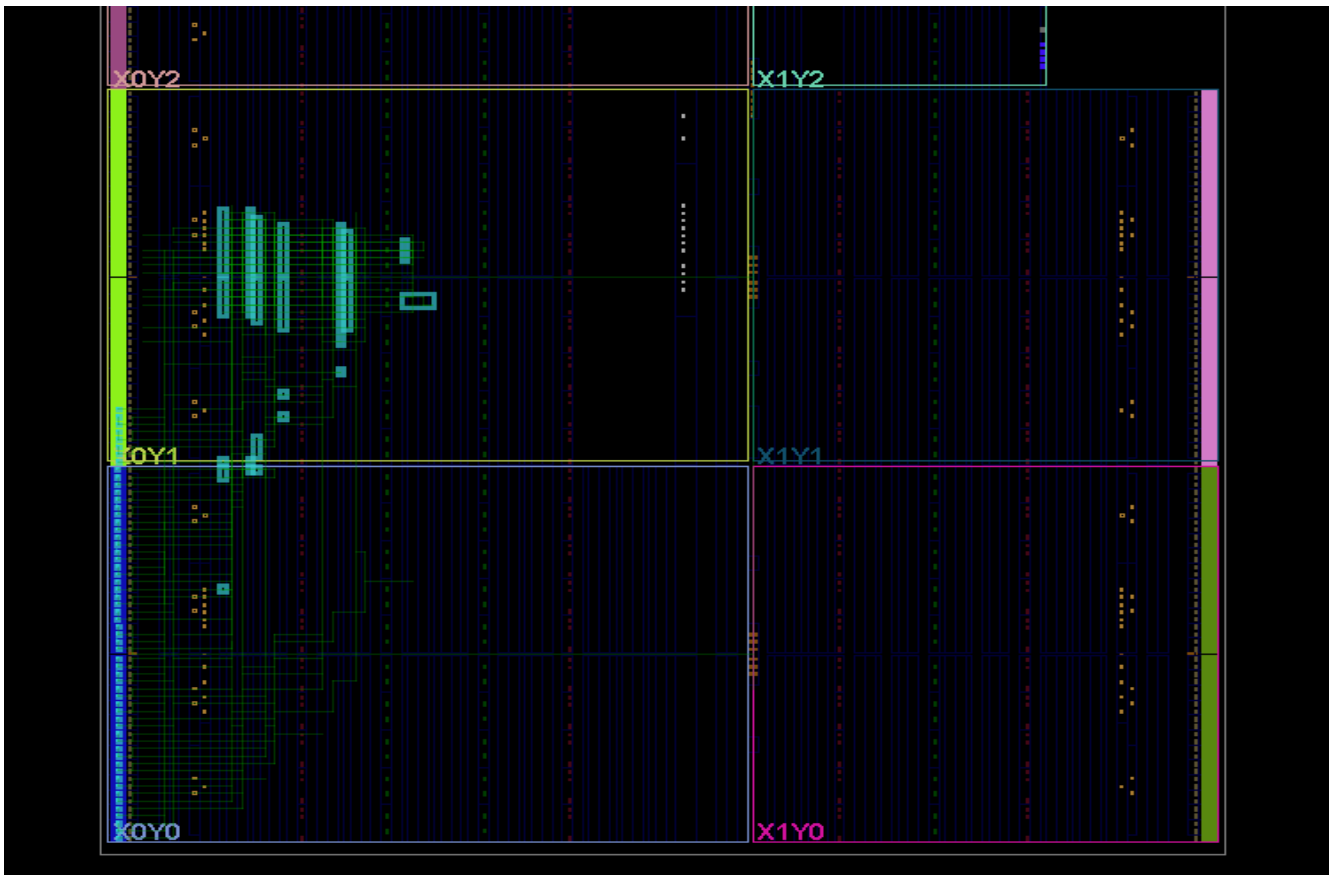
Il dispositivo utilizzato per implementare il circuito progettato è un FPGA AMD Xilinx Kintex 7 (xc7k70tfbv676-1).

Kintex-7 FPGA Feature Summary

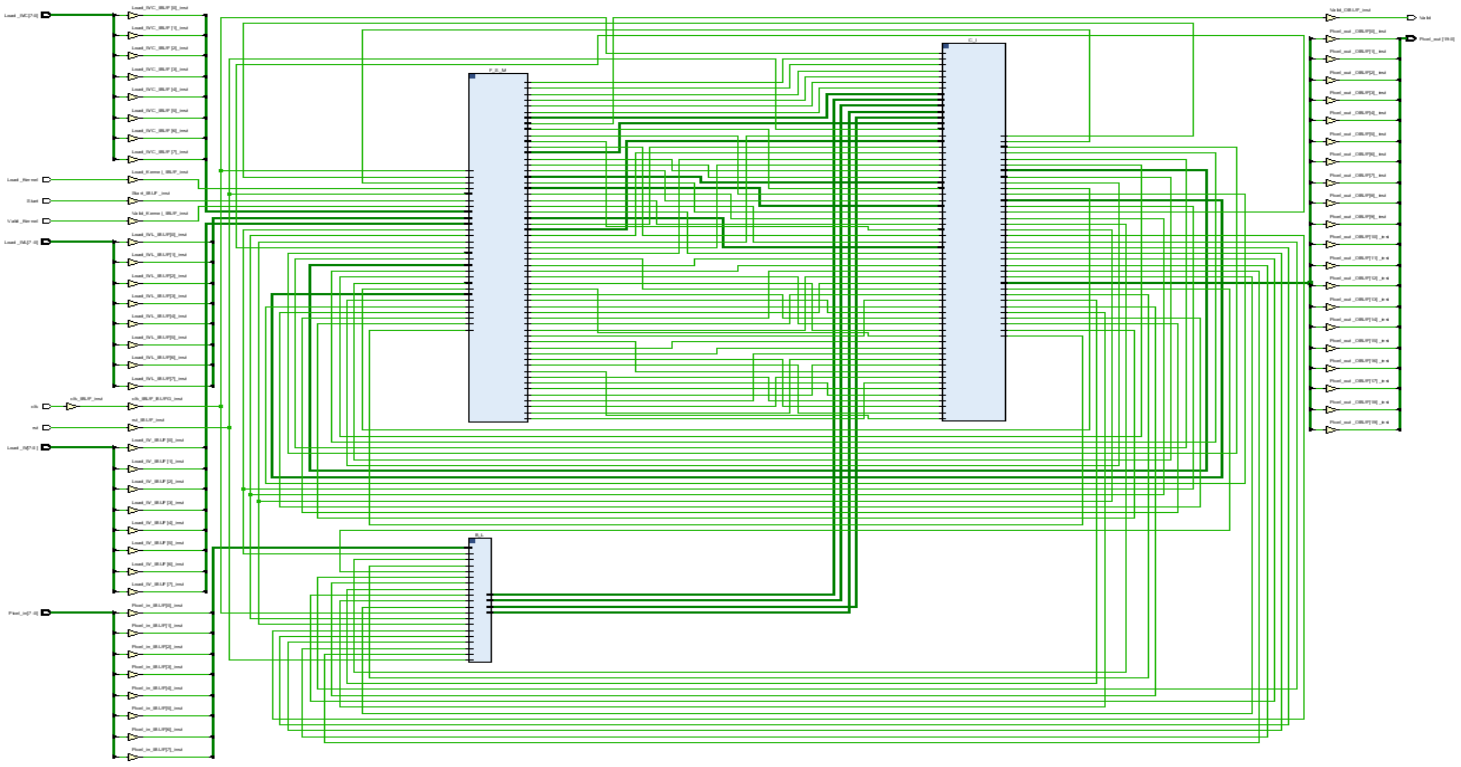
Device	Logic Cells	Configurable Logic Blocks (CLBs)		DSP Slices[2]	Block RAM Blocks[3]			CMTs[4]	PCIe[5]	GTXs	XADC Blocks	Total I/O Banks[6]	Max User I/O[7]
		Slices[1]	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)						
XC7K70T	65,600	10,250	838	240	270	135	4,860	6	1	8	1	6	300
XC7K160T	162,240	25,350	2,188	600	650	325	11,700	8	1	8	1	8	400
XC7K325T	326,080	50,950	4,000	840	890	445	16,020	10	1	16	1	10	500
XC7K355T	356,160	55,650	5,088	1,440	1,430	715	25,740	6	1	24	1	6	300
XC7K410T	406,720	63,550	5,663	1,540	1,590	795	28,620	10	1	16	1	10	500
XC7K420T	416,960	65,150	5,938	1,680	1,670	835	30,060	8	1	32	1	8	400
XC7K480T	477,760	74,650	6,788	1,920	1,910	955	34,380	8	1	32	1	8	400



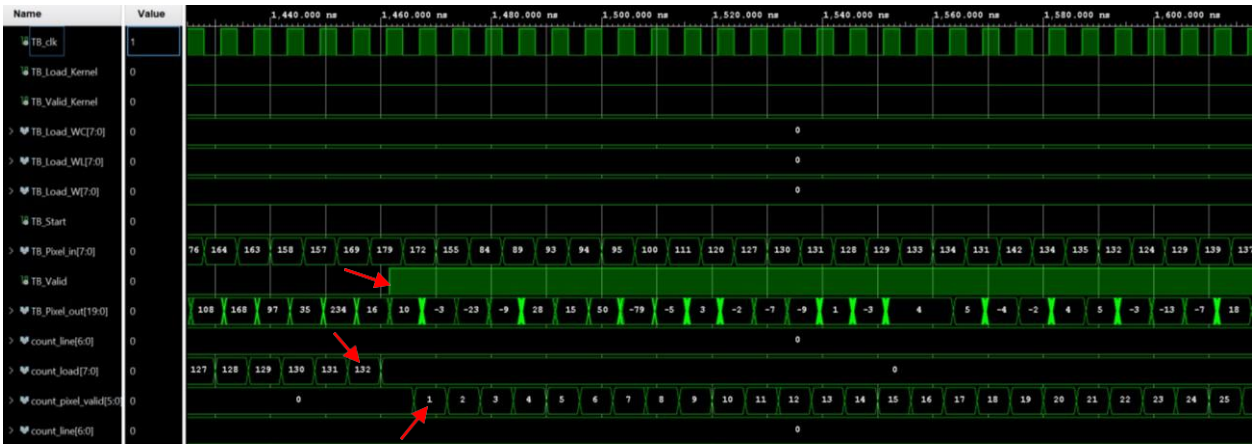
Rappresentazione dispositivo



Device

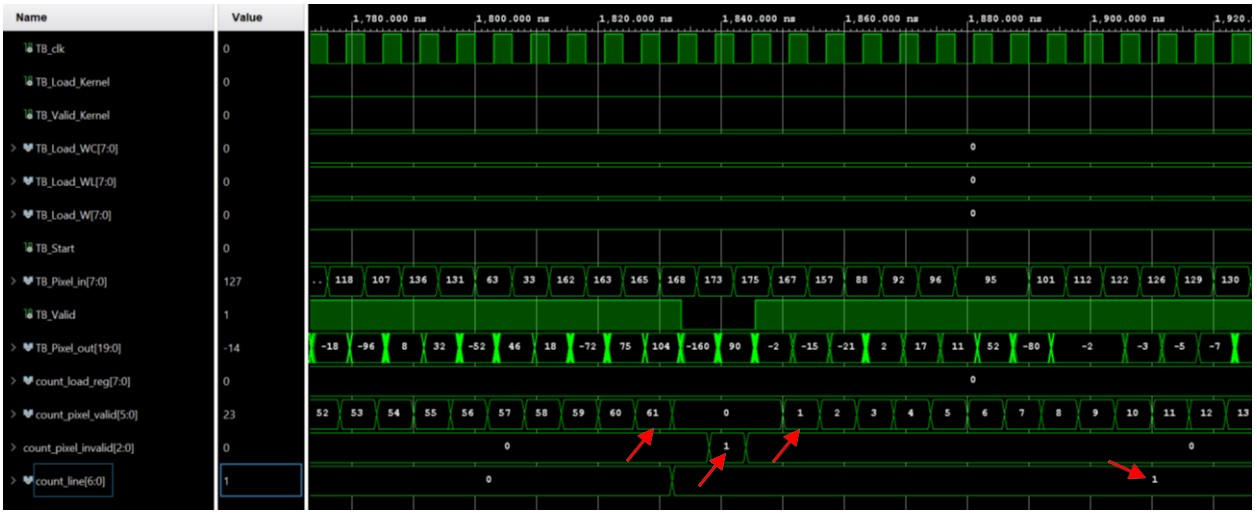


Schematic Post-Implementazione



Simulazione Timing Post-Implementazione: Primo Valore Valido

Quando il contatore count_load raggiunge il valore 132, l'FSM passa dallo stato Load a Pixel_Valid. In questo momento, il segnale Valid si attiva in corrispondenza del primo valore valido, mentre il contatore count_pixel_valid inizia a registrare il numero di pixel validi.



Simulazione Timing Post-Implementazione: Gestione Bordi

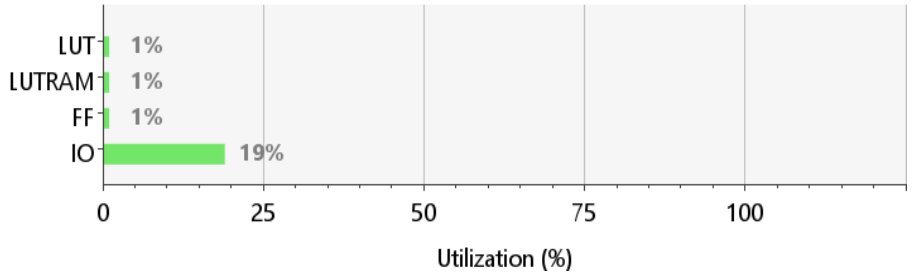
Nella Waveform generata da Vivado, si osserva che quando il contatore count_pixel_valid raggiunge il valore 61, l'FSM passa dallo stato Pixel_Valid a Pixel_Invalid, avviando il conteggio del contatore count_pixel_invalid. Quest'ultimo conta fino a 1, poiché il segnale Valid deve rimanere basso per due cicli di clock, corrispondenti ai pixel non validi dei bordi. Successivamente, l'FSM torna nello stato Pixel_Valid e il contatore count_pixel_valid, già azzerato in precedenza, ricomincia a registrare il numero di pixel validi.
La simulazione Timing post-implementazione viene effettuata con un periodo di clock pari al constraint imposto, ovvero 6 ns.

6 Report e conclusioni

Analisi delle risorse utilizzate:

- LUTRAM: queste risorse sono impiegate per gli Shift-Register della Buffer Line. Poiché l'ingresso è a 8 bit e sono presenti due Shift Register di 61 posizioni ciascuno, è necessario utilizzare due LUTRAM in cascata per ogni shift register da 61 posizioni, dato che ogni singola LUTRAM può essere configurata al massimo con 32 posizioni. Di conseguenza, per ogni bit di ingresso servono 4 LUTRAM, e per l'intera Buffer Line sono necessarie 32 LUTRAM.

Resource	Utilization	Available	Utilization %
LUT	517	41000	1.26
LUTRAM	32	13400	0.24
FF	217	82000	0.26
IO	57	300	19.00



Name	Slice LUTs (41000)	Slice Registers (82000)	Slice (10250)	LUT as Logic (41000)	LUT as Memory (13400)	Bonded IOB (300)	BUFGCTRL (32)
N Top_Image_Elaboration	517	217	149	485	32	57	1
B_L (Buffer_Line)	95	64	43	63	32	0	0
C_I (Convolutore_Isotropico)	375	100	120	375	0	0	0
F_S_M (FSM)	49	53	36	49	0	0	0

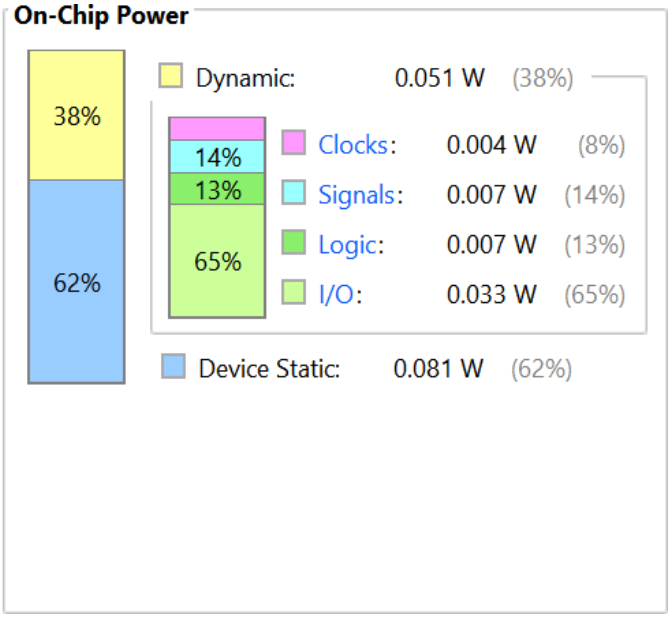
- Il numero di **I/O** utilizzati coincide con quanto descritto nel codice VHDL:
  - 8 per l'input dei pixel dell'immagine,
  - 1 per il segnale Valid,
  - 1 per il segnale di clock,
  - 24 per caricare i 3 coefficienti del filtro,
  - 20 per l'output dei pixel ottenuti,
  - 3 per segnali di controllo (Start, Load_Kernel e Valid_Kernel).

- **LUT e FF:** Per queste risorse, è necessario esaminare i report specifici di ogni blocco per comprendere esattamente come e dove vengono utilizzate. Nei report di potenza si può vedere che è stato ottenuto un livello di confidenza High, il quale ci garantisce che Vivado sia in grado di stimare il consumo energetico di tutti i blocchi implementati. Questo risultato è possibile grazie al file in formato SAIF (Switching Activity Interchange Format) fornito da Vivado, dove sono registrate tutte le attività di commutazioni dei segnali del sistema durante una simulazione Post-Implementation. Dal report di potenza risulta che la potenza totale è di 132 mW, suddivisa in 81 mW per la parte statica del dispositivo e 51 mW per la parte dinamica, che include il consumo di tutti i blocchi implementati.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.132 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25,2°C**
Thermal Margin:	59,8°C (31,5 W)
Effective ϑJA:	1,9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

Launch Power Constraint Advisor to find and fix invalid switching activity



Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0,682 ns	Worst Hold Slack (WHS):	0,097 ns	Worst Pulse Width Slack (WPWS):	2,220 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	262	Total Number of Endpoints:	262	Total Number of Endpoints:	250

Name	Waveform	Period (ns)	Frequency (MHz)
CLK	{0.000 3.000}	6.000	166.667

Il report sul segnale di clock mostra che la frequenza raggiunta è di 166,667 MHz, che corrisponde ad un periodo di 6 ns. Inoltre, la schermata di Vivado presenta un riepilogo dettagliato del timing del circuito implementato, dove tutti i parametri, sia di setup che di hold, risultano positivi. Questo conferma che non sono state rilevate violazioni dei requisiti temporali.